Министерство образования и науки РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Омский государственный технический университет»

Кафедра «Информатика и вычислительная техника»

Отчёт по лабораторной работе № 3
по дисциплине
«Экспертные системы и искусственный интеллект»

Выполнил
Студент гр. ИВТ-223
Пономарев М.И. _____
(подп., дата)

Проверил
старший преподаватель каф. ИВТ
Нигрей А.А. _____
(подп., дата)

Омск, 2025

Задание:

С помощью любого языка программирования высокого уровня реализовать программное обеспечение, реализующее методы искусственного интеллекта.

Результаты выполнения:

Программное обеспечение было реализовано на языке программирования высокого уровня Java и ее библиотеках. Весь воссозданный код прикрепляется в приложении к отчету. В программном обеспечении реализован генетический алгоритм, который демонстрирует симуляцию эволюции первых людей на основе их первоначальных характеристик и мутаций в процессе скрещивания между особями на протяжении n-го количества лет.

Для большей наглядности программное обеспечение имеет в себе встроенные функции для рассмотрения данных конкретной особи, а конкретно: характеристики данной особи, родительских особей, братьев и сестер, а также всего родственного древа конкретной особи, от основателей и до дальних потомков.

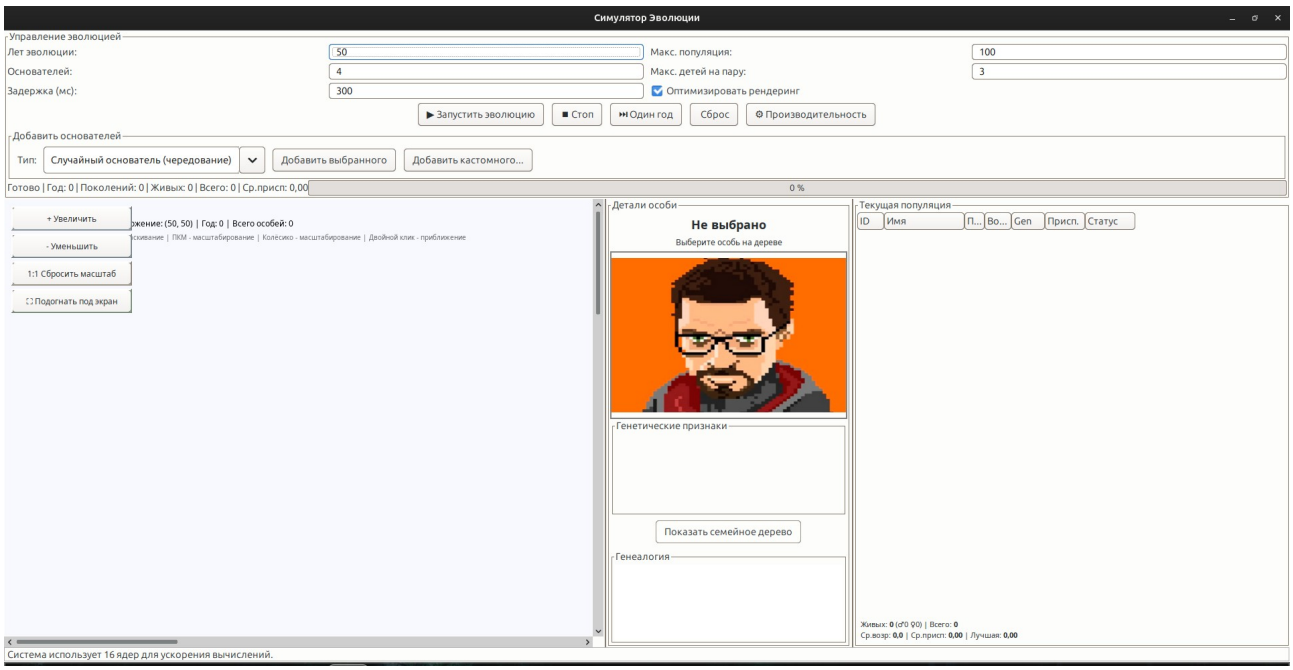


Рисунок 1 — Общий вид программного обеспечения.

Интерфейс состоит из нескольких разделов управления: управление эволюцией, где задаются параметры длительности и разные случайные переменные, а также добавлена система для ускорения работы приложения; добавить основателей, панель для добавления необходимого количества особей 0-го поколения (основателей), с необходимыми корректировками по желанию; панель отображения всех особей по поколениям; детали особи, где можно рассмотреть конкретную особь с ее параметрами, информацией о ближайших родственниках и ее генетическое дерево; текущая популяция, список со всеми особями в одной таблице.

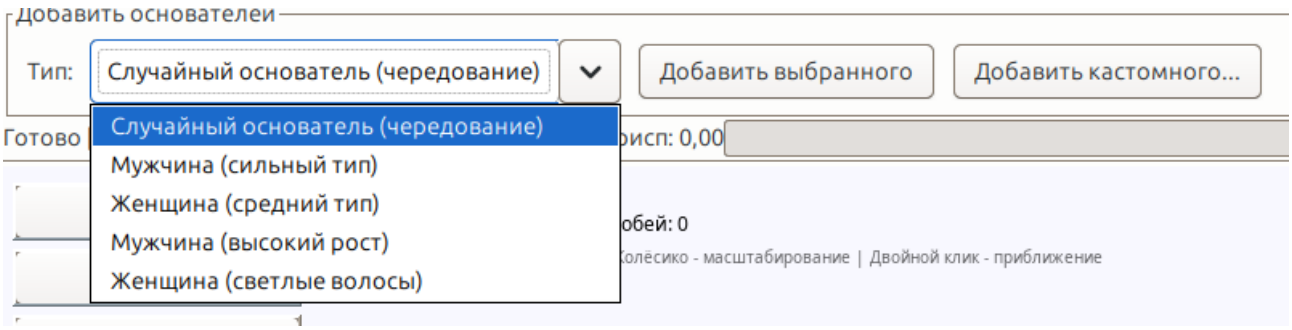


Рисунок 2 — Раздел для добавления основателей.

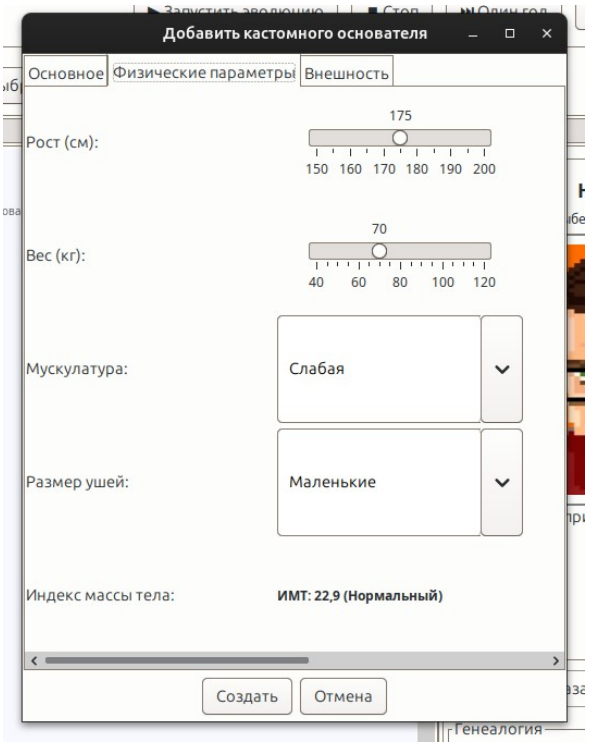


Рисунок 3 — Подраздел для добавления особи по предпочтению пользователя.

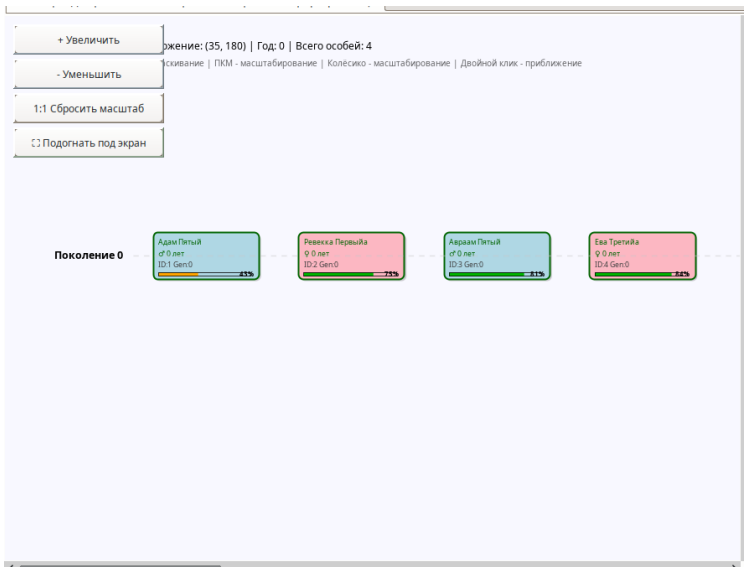


Рисунок 4 — Панель отображения всех особей.

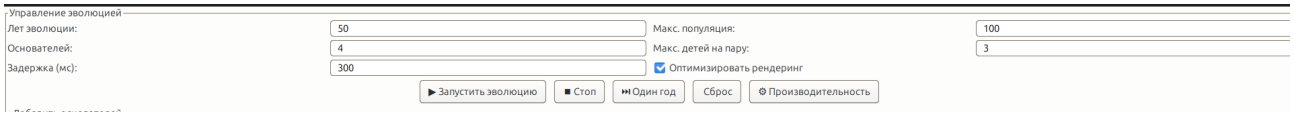


Рисунок 5 — Панель управления эволюцией.

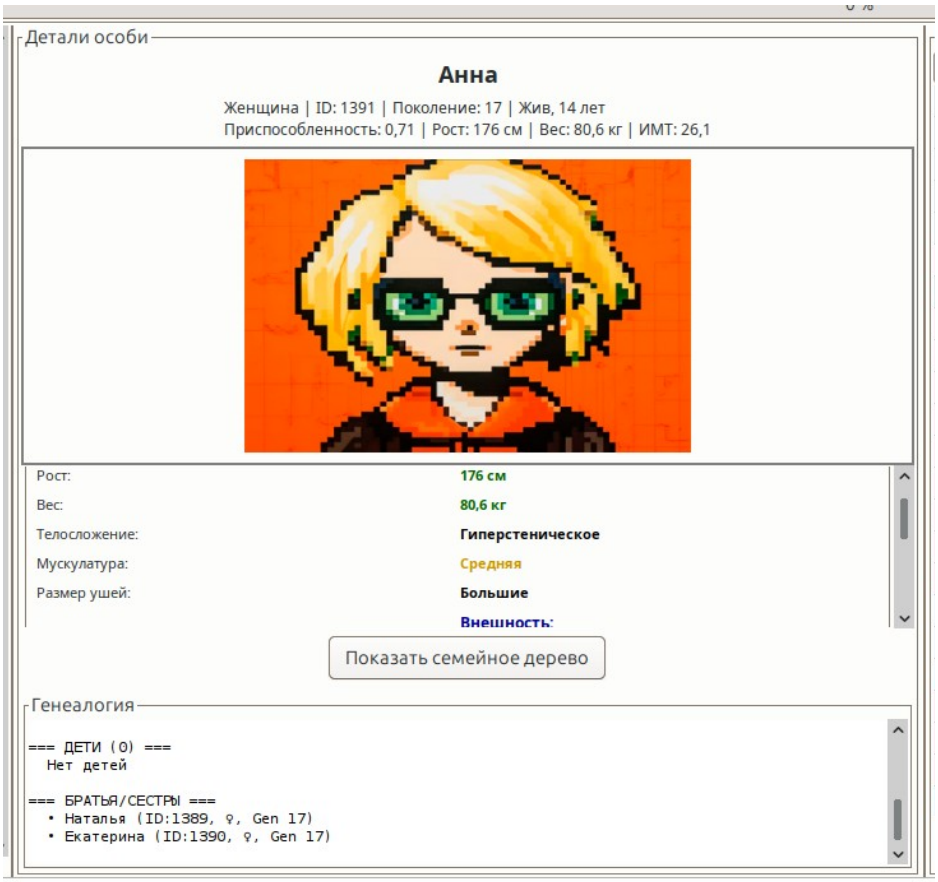


Рисунок 6 — Детали конкретной особи.



Рисунок 7 — Список текущей популяции.

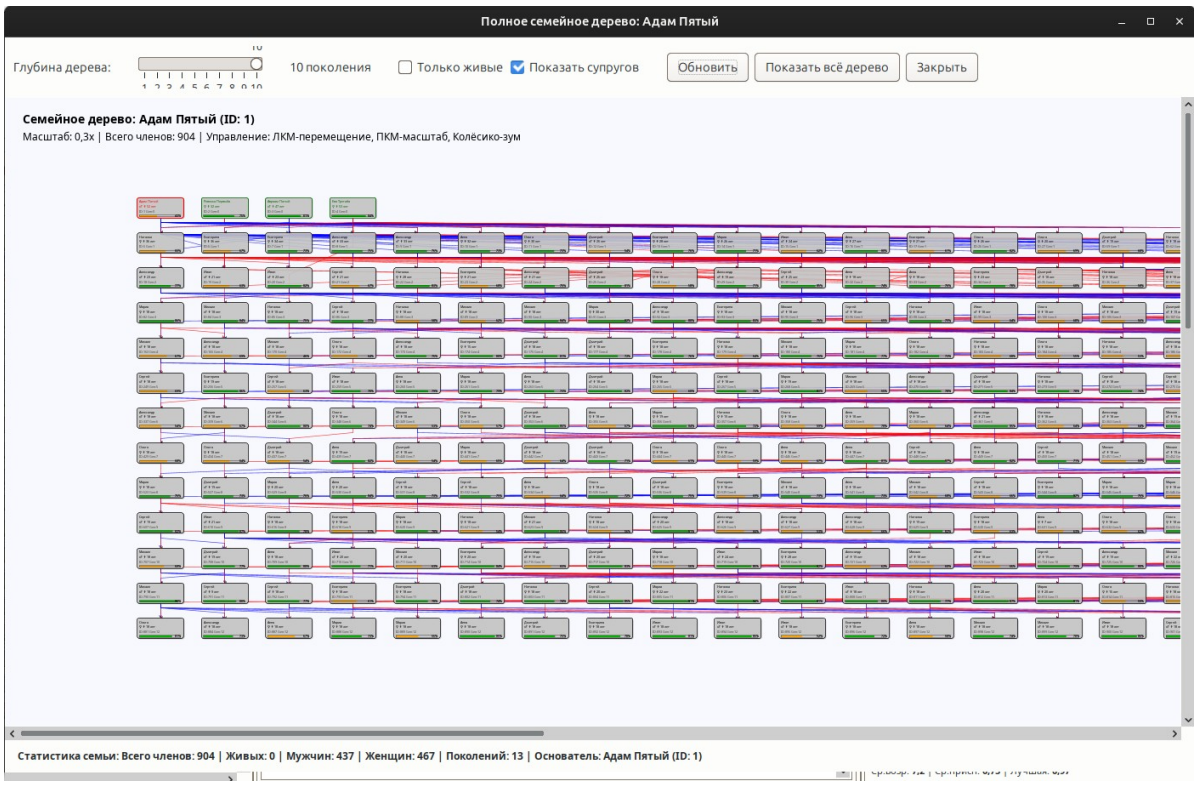


Рисунок 8 — Семейное древо конкретной особи.

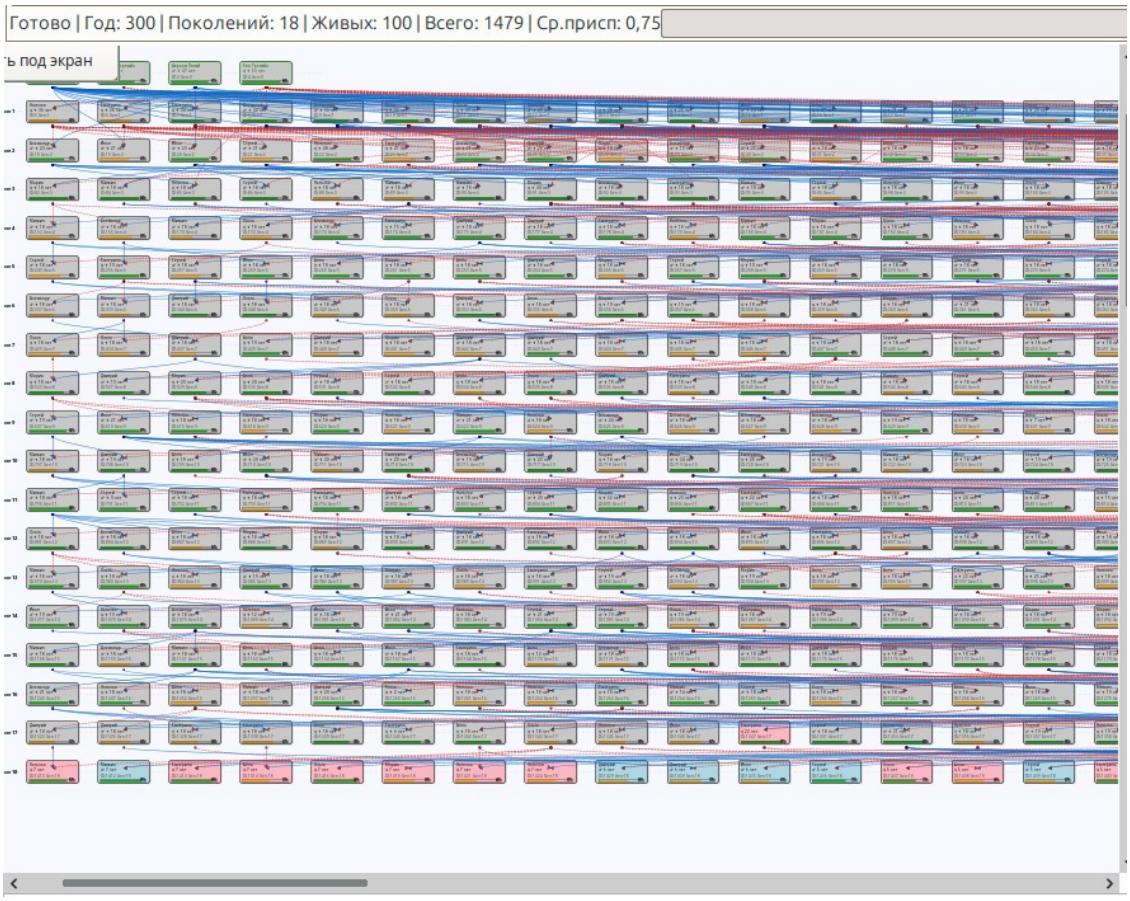


Рисунок 9 — Генетическое дерево всех особей после 300 лет развития.

Заключение

В ходе данной лабораторной работы был разработан генетический алгоритм с разными исходами, которые зависят от характеристик особей и случайных вероятностей различных исходов, на языке высокого уровня Java.

Все ключевые параметры вынесены в настраиваемые поля интерфейса, что позволяет экспериментировать с размерами популяции, скоростью мутации, типами операторов и числом поколений без изменения кода.

В результате получена рабочая программа для моделирования эволюции популяции и изучения влияния параметров на её поведение. Интерфейс позволяет запускать эксперименты и просматривать результаты в удобном виде.

Приложения

Файл EvolutionSimulator.java

Код:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
public class EvolutionSimulator {
    private static EvolutionEngine engine;
    public static void main(String[] args) {
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception e) {
            e.printStackTrace();
        }
        System.setProperty("sun.java2d.opengl", "true");
        System.setProperty("sun.java2d.d3d", "true");
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Симулятор Эволюции");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(1400, 850);
            engine = new EvolutionEngine();
            ControlPanel controlPanel = new ControlPanel(engine);
            GenealogyTreePanel treePanel = new GenealogyTreePanel(engine);
            DetailsPanel detailsPanel = new DetailsPanel(engine);
            PopulationPanel populationPanel = new PopulationPanel(engine);
            treePanel.setOptimizeRendering(controlPanel.isOptimizeRenderEnabled());
            controlPanel.getOptimizeRenderCheck().addActionListener(e -> {
                treePanel.setOptimizeRendering(controlPanel.isOptimizeRenderEnabled());
            });
            frame.setLayout(new BorderLayout());
            frame.add(controlPanel, BorderLayout.NORTH);
            JSplitPane centerSplit = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT,
```

```
                new JScrollPane(treePanel), detailsPanel);
        centerSplit.setDividerLocation(700);
        centerSplit.setResizeWeight(0.7);
        JSplitPane mainSplit = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT,
                centerSplit, populationPanel);
        mainSplit.setDividerLocation(900);
        mainSplit.setResizeWeight(0.7);
        frame.add(mainSplit, BorderLayout.CENTER);
                JLabel statusBar = new JLabel(" Готово. Добавьте основателей и запустите эволюцию.");
        statusBar.setBorder(BorderFactory.createEtchedBorder());
        frame.add(statusBar, BorderLayout.SOUTH);
        frame.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                engine.shutdown();
            }
            @Override
            public void windowClosed(WindowEvent e) {
                engine.shutdown();
            }
        });
        frame.setVisible(true);
        SwingUtilities.invokeLater(() -> {
            int cores = Runtime.getRuntime().availableProcessors();
            if (cores > 1) {
                        statusBar.setText(" Система использует " + cores + " ядер для ускорения вычислений.");
            }
        });
    });
  }
}
```

Файл EvolutionEngine.java

Код:

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Random;
import java.util.Set;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.Callable;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.Collectors;
import javax.swing.event.ChangeEvent;
import javax.swing.event.ChangeListener;
import javax.swing.SwingUtilities;
class EvolutionEngine {
    private List<Human> allPeople = new CopyOnWriteArrayList<>();
    private List<Human> alivePeople = new CopyOnWriteArrayList<>();
    private int currentYear = 0;
    private Human selectedHuman = null;
    private ThreadLocal<Random> rand = ThreadLocal.withInitial(Random::new);
    private List<ChangeListener> listeners = new CopyOnWriteArrayList<>();
    private int maxPopulation = 100;
    private double mutationRate = 0.1;
    private int maxChildrenPerCouple = 3;
    private volatile boolean evolutionRunning = false;
```

```
private int maleFoundersAdded = 0;
private int femaleFoundersAdded = 0;
private ExecutorService executorService;
private int threadPoolSize = Runtime.getRuntime().availableProcessors();
private AtomicInteger nextId = new AtomicInteger(1);
private final Object updateLock = new Object();
public EvolutionEngine() {
    executorService = Executors.newFixedThreadPool(threadPoolSize);
}
public void shutdown() {
    if (executorService != null && !executorService.isShutdown()) {
        executorService.shutdown();
        try {
            executorService.awaitTermination(5, TimeUnit.SECONDS);
        } catch (InterruptedException e) {
            executorService.shutdownNow();
        }
    }
}
public void initializePopulation(int founderCount) {
    synchronized (updateLock) {
        allPeople.clear();
        alivePeople.clear();
        currentYear = 0;
        selectedHuman = null;
        maleFoundersAdded = 0;
        femaleFoundersAdded = 0;
        nextId.set(1);
        List<Callable<Human>> tasks = new ArrayList<>();
        for (int i = 0; i < founderCount; i++) {
            final boolean needMale = needsMaleFounder();
            tasks.add(() -> createRandomFounderWithGender(needMale));
        }
        try {
```

```
            List<Future<Human>> futures = executorService.invokeAll(tasks);
            for (Future<Human> future : futures) {
                Human founder = future.get();
                allPeople.add(founder);
                alivePeople.add(founder);
                if (founder.isMale) maleFoundersAdded++;
                else femaleFoundersAdded++;
            }
        } catch (InterruptedException | ExecutionException e) {
            e.printStackTrace();
        }
    }
    notifyListeners();
}
private Human createRandomFounderWithGender(boolean needMale) {
    String[] maleNames = {"Адам", "Ной", "Авраам", "Моисей", "Давид",
            "Иван", "Петр", "Алексей", "Сергей", "Дмитрий"};
    String[] femaleNames = {"Ева", "Сарра", "Ревекка", "Рахиль", "Лия",
            "Анна", "Мария", "Елена", "Ольга", "Наталья"};
    String[] surnames = {"Иванов", "Петров", "Сидоров", "Смирнов", "Кузнецов"};
    Random localRand = rand.get();
    boolean isMale = needMale;
    String name;
    if (isMale) {
        name = maleNames[localRand.nextInt(maleNames.length)] + " " +
                surnames[localRand.nextInt(surnames.length)];
    } else {
        name = femaleNames[localRand.nextInt(femaleNames.length)] + " " +
                surnames[localRand.nextInt(surnames.length)] + "a";
    }
    return new Human(name, isMale, 0, null, null, localRand, nextId.getAndIncrement());
}
public void addRandomFounder() {
    boolean needMale = needsMaleFounder();
```

```
        Human founder = createRandomFounderWithGender(needMale);
        synchronized (updateLock) {
            allPeople.add(founder);
            alivePeople.add(founder);
            if (founder.isMale) maleFoundersAdded++;
            else femaleFoundersAdded++;
        }
        notifyListeners();
    }
    public void addCustomFounder(String name, boolean isMale, Map<String, Object> traits) {
                    Human founder = new Human(name, isMale, 0, null, null, rand.get(),
nextId.getAndIncrement());
        if (traits != null) {
            for (Map.Entry<String, Object> entry : traits.entrySet()) {
                if (entry.getKey() != null && entry.getValue() != null) {
                    founder.traits.put(entry.getKey(), entry.getValue());
                }
            }
            founder.calculateFitness();
        }
        synchronized (updateLock) {
            allPeople.add(founder);
            alivePeople.add(founder);
            if (isMale) maleFoundersAdded++;
            else femaleFoundersAdded++;
        }
        notifyListeners();
    }
    public boolean needsMaleFounder() {
        return maleFoundersAdded <= femaleFoundersAdded;
    }
    public void evolveOneYear() {
        if (alivePeople.isEmpty()) return;
        currentYear++;
```

```
List<Future<Void>> agingFutures = new ArrayList<>();
List<Human> aliveCopy = new ArrayList<>(alivePeople);
int batchSize = Math.max(1, aliveCopy.size() / threadPoolSize);
for (int i = 0; i < aliveCopy.size(); i += batchSize) {
    final int start = i;
    final int end = Math.min(i + batchSize, aliveCopy.size());
    final List<Human> batch = aliveCopy.subList(start, end);
    agingFutures.add(executorService.submit(() -> {
        List<Human> deadInBatch = new ArrayList<>();
        Random localRand = rand.get();
        for (Human human : batch) {
            human.ageOneYear(localRand);
            if (!human.alive) {
                deadInBatch.add(human);
            }
        }
        if (!deadInBatch.isEmpty()) {
            synchronized (updateLock) {
                alivePeople.removeAll(deadInBatch);
            }
        }
        return null;
    }));
}
waitForFutures(agingFutures);
List<Human> males = new ArrayList<>();
List<Human> females = new ArrayList<>();
Random localRand = rand.get();
for (Human human : alivePeople) {
    if (human.canReproduce()) {
        if (localRand.nextDouble() < 0.7) {
            if (human.isMale) {
                males.add(human);
            } else {
```

```
                females.add(human);
            }
        }
    }
}
Collections.shuffle(males, localRand);
Collections.shuffle(females, localRand);
List<Human> newChildren = Collections.synchronizedList(new ArrayList<>());
int pairs = Math.min(males.size(), females.size());
List<Future<Void>> reproductionFutures = new ArrayList<>();
int reproductionBatchSize = Math.max(1, pairs / threadPoolSize);
for (int i = 0; i < pairs; i += reproductionBatchSize) {
    final int start = i;
    final int end = Math.min(i + reproductionBatchSize, pairs);
    reproductionFutures.add(executorService.submit(() -> {
        Random threadRand = rand.get();
        List<Human> childrenBatch = new ArrayList<>();
        for (int j = start; j < end; j++) {
            Human male = males.get(j);
            Human female = females.get(j);
            int childGeneration = Math.max(male.generation, female.generation) + 1;
            double maleFertility = 1.0 - Math.max(0, (male.age - 35) / 20.0);
            double femaleFertility = 1.0 - Math.max(0, (female.age - 30) / 20.0);
            double fertility = (maleFertility + femaleFertility) / 2.0;
            int childrenCount = threadRand.nextInt(maxChildrenPerCouple + 1);
            for (int k = 0; k < childrenCount; k++) {
                if (threadRand.nextDouble() < fertility * 0.7) {
                        String[] maleNames = {"Александр", "Дмитрий", "Михаил", "Иван",
"Сергей"};
                    String[] femaleNames = {"Анна", "Мария", "Екатерина", "Ольга", "Наталья"};
                    boolean childIsMale = threadRand.nextBoolean();
                    String childName = childIsMale ?
                        maleNames[threadRand.nextInt(maleNames.length)] :
                        femaleNames[threadRand.nextInt(femaleNames.length)];
```

```
                Human child = new Human(childName, childIsMale,
                        childGeneration, male, female, threadRand, nextId.getAndIncrement());
                childrenBatch.add(child);
            }
        }
    }
    if (!childrenBatch.isEmpty()) {
        synchronized (updateLock) {
            newChildren.addAll(childrenBatch);
        }
    }
    return null;
  }));
}
waitForFutures(reproductionFutures);
if (!newChildren.isEmpty()) {
  synchronized (updateLock) {
    for (Human child : newChildren) {
      allPeople.add(child);
      alivePeople.add(child);
      if (child.father != null) child.father.children.add(child);
      if (child.mother != null) child.mother.children.add(child);
    }
  }
}
if (alivePeople.size() > maxPopulation) {
  List<Human> sortedAlive = new ArrayList<>(alivePeople);
  List<Human> sortedForRemoval = sortedAlive.parallelStream()
      .filter(h -> h.age >= 18)
      .sorted((h1, h2) -> {
        double score1 = h1.age / 100.0 + (1 - h1.fitness);
        double score2 = h2.age / 100.0 + (1 - h2.fitness);
        return Double.compare(score2, score1);
      })
```

```
                .collect(Collectors.toList());
        int toRemove = Math.max(0, alivePeople.size() - maxPopulation);
        List<Human> toRemoveList = new ArrayList<>();
        int maxToRemove = Math.max(1, alivePeople.size() / 10);
        toRemove = Math.min(toRemove, maxToRemove);
        for (int i = 0; i < toRemove && i < sortedForRemoval.size(); i++) {
            toRemoveList.add(sortedForRemoval.get(i));
        }
        synchronized (updateLock) {
            for (Human removed : toRemoveList) {
                alivePeople.remove(removed);
                removed.alive = false;
                removed.deathAge = removed.age;
            }
        }
    }
    SwingUtilities.invokeLater(this::notifyListeners);
}
private void waitForFutures(List<Future<Void>> futures) {
    for (Future<Void> future : futures) {
        try {
            future.get();
        } catch (InterruptedException | ExecutionException e) {
            e.printStackTrace();
        }
    }
}
public void evolveYears(int years) {
    evolutionRunning = true;
    CompletableFuture.runAsync(() -> {
        for (int i = 0; i < years && evolutionRunning; i++) {
            long startTime = System.currentTimeMillis();
            evolveOneYear();
            try {
```

```
                long elapsed = System.currentTimeMillis() - startTime;
                long sleepTime = Math.max(50, 300 - elapsed);
                Thread.sleep(sleepTime);
            } catch (InterruptedException e) {
                break;
            }
        }
        evolutionRunning = false;
        SwingUtilities.invokeLater(this::notifyListeners);
    }, executorService);
}
public void stopEvolution() {
    evolutionRunning = false;
}
public void selectHuman(Human human) {
    selectedHuman = human;
    SwingUtilities.invokeLater(this::notifyListeners);
}
public List<Human> getFamilyTree(Human founder, int maxDepth) {
    List<Human> family = new ArrayList<>();
    if (founder == null) return family;
    Set<Human> visited = new HashSet<>();
    getAllDescendants(founder, family, visited, 0, maxDepth);
    addSpouses(family, visited);
    return family;
}
public List<Human> getFamilyTree(Human founder) {
    return getFamilyTree(founder, Integer.MAX_VALUE);
}
public List<Human> getAllDescendants(Human founder) {
    return getFamilyTree(founder, Integer.MAX_VALUE);
}
private void getAllDescendants(Human person, List<Human> family,
                               Set<Human> visited, int depth, int maxDepth) {
```

```
    if (person == null || visited.contains(person) || depth > maxDepth) return;
    family.add(person);
    visited.add(person);
    for (Human child : person.children) {
        if (!visited.contains(child)) {
            getAllDescendants(child, family, visited, depth + 1, maxDepth);
        }
    }
}
private void addSpouses(List<Human> family, Set<Human> visited) {
    Set<Human> spouses = new HashSet<>();
    for (Human person : family) {
        for (Human child : person.children) {
            Human spouse = null;
            if (child.father == person) {
                spouse = child.mother;
            } else if (child.mother == person) {
                spouse = child.father;
            }
            if (spouse != null && !visited.contains(spouse) && !family.contains(spouse)) {
                spouses.add(spouse);
            }
        }
    }
    family.addAll(spouses);
}
private Human findSpouse(Human person) {
    if (person == null) return null;
    for (Human child : person.children) {
        if (person.isMale && child.mother != null) {
            return child.mother;
        } else if (!person.isMale && child.father != null) {
            return child.father;
        }
```

```
    }
    return null;
}
public List<Human> getFounders() {
    synchronized (updateLock) {
        return allPeople.stream()
                .filter(Human::isFounder)
                .collect(Collectors.toList());
    }
}
public List<Human> getAllPeople() {
    synchronized (updateLock) {
        return new ArrayList<>(allPeople);
    }
}
public List<Human> getAlivePeople() {
    synchronized (updateLock) {
        return new ArrayList<>(alivePeople);
    }
}
public int getCurrentYear() { return currentYear; }
public Human getSelectedHuman() { return selectedHuman; }
public int getTotalPeopleCount() {
    synchronized (updateLock) {
        return allPeople.size();
    }
}
public int getAlivePeopleCount() {
    synchronized (updateLock) {
        return alivePeople.size();
    }
}
public boolean isEvolutionRunning() { return evolutionRunning; }
public double getAverageFitness() {
```

```
    List<Human> aliveCopy;
    synchronized (updateLock) {
        aliveCopy = new ArrayList<>(alivePeople);
    }
    if (aliveCopy.isEmpty()) return 0;
    return aliveCopy.parallelStream()
            .mapToDouble(h -> h.fitness)
            .average()
            .orElse(0);
}
public double getBestFitness() {
    List<Human> aliveCopy;
    synchronized (updateLock) {
        aliveCopy = new ArrayList<>(alivePeople);
    }
    if (aliveCopy.isEmpty()) return 0;
    return aliveCopy.parallelStream()
            .mapToDouble(h -> h.fitness)
            .max()
            .orElse(0);
}
public int getMaxGeneration() {
    List<Human> allCopy;
    synchronized (updateLock) {
        allCopy = new ArrayList<>(allPeople);
    }
    return allCopy.parallelStream()
            .mapToInt(h -> h.generation)
            .max()
            .orElse(0);
}
public void setMaxPopulation(int max) { this.maxPopulation = max; }
public void setMutationRate(double rate) { this.mutationRate = rate; }
public void setChildrenPerCouple(int children) { this.maxChildrenPerCouple = children; }
```

```
    public void setThreadPoolSize(int size) {
        if (size > 0 && size <= 32) {
            this.threadPoolSize = size;
            if (executorService != null) {
                executorService.shutdown();
                executorService = Executors.newFixedThreadPool(size);
            }
        }
    }
    public void addChangeListener(ChangeListener listener) {
        listeners.add(listener);
    }
    private void notifyListeners() {
        ChangeEvent event = new ChangeEvent(this);
        for (ChangeListener listener : listeners) {
            listener.stateChanged(event);
        }
    }
}
```

Файл ControlPanel.java

Код:

```
import javax.swing.*;
import javax.swing.event.*;
import java.awt.*;
import java.awt.event.*;
import java.util.*;
class ControlPanel extends JPanel {
    private EvolutionEngine engine;
    private JTextField yearsField;
    private JTextField populationField;
    private JTextField foundersField;
    private JButton startButton;
    private JButton stopButton;
```

```
private JButton stepButton;
private JButton resetButton;
private JButton addFounderButton;
private JButton addCustomFounderButton;
private JLabel statusLabel;
private JProgressBar progressBar;
private JComboBox<String> founderTypeCombo;
private javax.swing.Timer statusTimer;
private JCheckBox optimizeRenderCheck;
public ControlPanel(EvolutionEngine engine) {
    this.engine = engine;
    setLayout(new BorderLayout());
    setBorder(BorderFactory.createTitledBorder("Управление эволюцией"));
    initializeComponents();
    statusTimer = new javax.swing.Timer(100, e -> updateStatus());
    statusTimer.start();
}
private void initializeComponents() {
    JPanel topPanel = new JPanel(new GridLayout(3, 4, 10, 5));
    topPanel.add(new JLabel("Лет эволюции:"));
    yearsField = new JTextField("50");
    topPanel.add(yearsField);
    topPanel.add(new JLabel("Макс. популяция:"));
    populationField = new JTextField("100");
    topPanel.add(populationField);
    topPanel.add(new JLabel("Основателей:"));
    foundersField = new JTextField("4");
    topPanel.add(foundersField);
    topPanel.add(new JLabel("Макс. детей на пару:"));
    JTextField childrenField = new JTextField("3");
    childrenField.addActionListener(e -> {
        try {
            engine.setChildrenPerCouple(Integer.parseInt(childrenField.getText()));
        } catch (NumberFormatException ex) {
```

```
        throw new RuntimeException(ex);
    }
});
topPanel.add(childrenField);
topPanel.add(new JLabel("Задержка (мс):"));
JTextField delayField = new JTextField("300");
delayField.addActionListener(e -> {
    try {
        JOptionPane.showMessageDialog(this,
                "Изменение задержки потребует перезапуска симуляции",
                "Информация", JOptionPane.INFORMATION_MESSAGE);
    } catch (NumberFormatException ex) {
        throw new RuntimeException(ex);
    }
});
topPanel.add(delayField);
optimizeRenderCheck = new JCheckBox("Оптимизировать рендеринг", true);
topPanel.add(optimizeRenderCheck);
topPanel.add(new JLabel(""));
JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 5));
startButton = new JButton("▶️ Запустить эволюцию");
stopButton = new JButton("⏹️ Стоп");
stepButton = new JButton("⏭️ Один год");
resetButton = new JButton("🔄 Сброс");
JButton perfButton = new JButton("⚙️ Производительность");
buttonPanel.add(startButton);
buttonPanel.add(stopButton);
buttonPanel.add(stepButton);
buttonPanel.add(resetButton);
buttonPanel.add(perfButton);
JPanel founderPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 5));
founderPanel.setBorder(BorderFactory.createTitledBorder("Добавить основателей"));
founderTypeCombo = new JComboBox<>(new String[]{
        "Случайный основатель (чередование)",
```

```
        "Мужчина (сильный тип)",
        "Женщина (средний тип)",
        "Мужчина (высокий рост)",
        "Женщина (светлые волосы)"
});
addFounderButton = new JButton("Добавить выбранного");
addCustomFounderButton = new JButton("Добавить кастомного...");
founderPanel.add(new JLabel("Тип:"));
founderPanel.add(founderTypeCombo);
founderPanel.add(addFounderButton);
founderPanel.add(addCustomFounderButton);
JPanel statusPanel = new JPanel(new BorderLayout());
statusLabel = new JLabel("Готово к работе");
progressBar = new JProgressBar();
progressBar.setStringPainted(true);
statusPanel.add(statusLabel, BorderLayout.WEST);
statusPanel.add(progressBar, BorderLayout.CENTER);
startButton.addActionListener(e -> startEvolution());
stopButton.addActionListener(e -> {
    engine.stopEvolution();
    statusTimer.stop();
    statusTimer.start();
});
stepButton.addActionListener(e -> engine.evolveOneYear());
resetButton.addActionListener(e -> resetSimulation());
perfButton.addActionListener(e -> showPerformanceDialog());
addFounderButton.addActionListener(e -> addFounder());
addCustomFounderButton.addActionListener(e -> addCustomFounder());
populationField.addActionListener(e -> {
    try {
        engine.setMaxPopulation(Integer.parseInt(populationField.getText()));
    } catch (NumberFormatException ex) {}
});
JPanel mainPanel = new JPanel(new BorderLayout());
```

```
    mainPanel.add(topPanel, BorderLayout.NORTH);
    mainPanel.add(buttonPanel, BorderLayout.CENTER);
    mainPanel.add(founderPanel, BorderLayout.SOUTH);
    add(mainPanel, BorderLayout.NORTH);
    add(statusPanel, BorderLayout.SOUTH);
}
private void startEvolution() {
    try {
        int years = Integer.parseInt(yearsField.getText());
        int founders = Integer.parseInt(foundersField.getText());
        int maxPop = Integer.parseInt(populationField.getText());

        if (engine.getAlivePeopleCount() == 0) {
            if (founders % 2 != 0) founders++;
            engine.initializePopulation(founders);
        }
        engine.setMaxPopulation(maxPop);
        engine.evolveYears(years);

    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this,
                "Проверьте правильность введенных чисел!",
                "Ошибка", JOptionPane.ERROR_MESSAGE);
    }
}
private void resetSimulation() {
    int result = JOptionPane.showConfirmDialog(this,
            "Вы уверены, что хотите сбросить симуляцию?\nВсе данные будут потеряны.",
            "Сброс симуляции", JOptionPane.YES_NO_OPTION);
    if (result == JOptionPane.YES_OPTION) {
        engine.stopEvolution();
        engine.initializePopulation(0);
        statusTimer.restart();
    }
```

```
}
private void showPerformanceDialog() {
    JDialog dialog = new JDialog((Frame)null, "Настройки производительности", true);
    dialog.setLayout(new BorderLayout());
    dialog.setSize(400, 250);
    dialog.setLocationRelativeTo(this);
    JPanel panel = new JPanel(new GridLayout(4, 2, 10, 10));
    panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
    int cores = Runtime.getRuntime().availableProcessors();
    panel.add(new JLabel("Количество потоков:"));
    JSlider threadSlider = new JSlider(1, cores * 2, Math.min(cores, 4));
    threadSlider.setMajorTickSpacing(2);
    threadSlider.setMinorTickSpacing(1);
    threadSlider.setPaintTicks(true);
    threadSlider.setPaintLabels(true);
    panel.add(threadSlider);
    JLabel threadLabel = new JLabel(threadSlider.getValue() + " потоков");
    panel.add(new JLabel("Рекомендуется: " + cores));
    panel.add(threadLabel);
    threadSlider.addChangeListener(e -> {
        threadLabel.setText(threadSlider.getValue() + " потоков");
    });
    panel.add(new JLabel("Размер батча:"));
    JComboBox<String> batchCombo = new JComboBox<>(new String[]{
            "Авто", "Мелкий (10)", "Средний (50)", "Крупный (100)"
    });
    batchCombo.setSelectedIndex(0);
    panel.add(batchCombo);
    JButton applyButton = new JButton("Применить");
    applyButton.addActionListener(e -> {
        try {
            int threads = threadSlider.getValue();
            engine.setThreadPoolSize(threads);
            String batchSize = (String) batchCombo.getSelectedItem();
```

```
            JOptionPane.showMessageDialog(dialog,
                "Настройки применены!\n" +
                    "Потоки: " + threads + "\n" +
                    "Батчинг: " + batchSize,
                "Успех", JOptionPane.INFORMATION_MESSAGE);
            dialog.dispose();
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(dialog,
                "Ошибка применения настроек: " + ex.getMessage(),
                "Ошибка", JOptionPane.ERROR_MESSAGE);
        }
    });
    JButton cancelButton = new JButton("Отмена");
    cancelButton.addActionListener(e -> dialog.dispose());
    JPanel buttonPanel = new JPanel(new FlowLayout());
    buttonPanel.add(applyButton);
    buttonPanel.add(cancelButton);
    dialog.add(panel, BorderLayout.CENTER);
    dialog.add(buttonPanel, BorderLayout.SOUTH);
    dialog.setVisible(true);
}
private void addFounder() {
    String type = (String) founderTypeCombo.getSelectedItem();
    Random rand = new Random();
    String[] maleNames = {"Адам", "Ной", "Авраам", "Моисей", "Давид"};
    String[] femaleNames = {"Ева", "Сарра", "Ревекка", "Рахиль", "Лия"};
    String[] surnames = {"Первый", "Второй", "Третий", "Четвертый", "Пятый"};
    boolean isMale = false;
    String name;
    switch (type) {
        case "Случайный основатель (чередование)":
            boolean needMale = engine.needsMaleFounder();
            isMale = needMale;
            if (needMale) {
```

```
            name = maleNames[rand.nextInt(maleNames.length)] + " " +
                    surnames[rand.nextInt(surnames.length)];
        } else {
            name = femaleNames[rand.nextInt(femaleNames.length)] + " " +
                    surnames[rand.nextInt(surnames.length)] + "a";
        }
        break;
    case "Мужчина (сильный тип)":
        isMale = true;
        name = maleNames[rand.nextInt(maleNames.length)] + " Сильный";
        break;
    case "Женщина (средний тип)":
        isMale = false;
        name = femaleNames[rand.nextInt(femaleNames.length)] + " Средняя";
        break;
    case "Мужчина (высокий рост)":
        isMale = true;
        name = maleNames[rand.nextInt(maleNames.length)] + " Высокий";
        break;
    case "Женщина (светлые волосы)":
        isMale = false;
        name = femaleNames[rand.nextInt(femaleNames.length)] + " Светлая";
        break;
    default:
        boolean defaultMale = engine.needsMaleFounder();
        isMale = defaultMale;
        if (defaultMale) {
            name = maleNames[rand.nextInt(maleNames.length)] + " " +
                    surnames[rand.nextInt(surnames.length)];
        } else {
            name = femaleNames[rand.nextInt(femaleNames.length)] + " " +
                    surnames[rand.nextInt(surnames.length)] + "a";
        }
}
```

```
Map<String, Object> traits = new HashMap<>();
switch (type) {
    case "Мужчина (сильный тип)":
        traits.put("Мускулатура", "Сильная");
        traits.put("Рост", 185.0);
        traits.put("Вес", 85.0);
        break;
    case "Женщина (средний тип)":
        traits.put("Рост", 165.0);
        traits.put("Вес", 60.0);
        traits.put("Цвет волос", "Русые");
        traits.put("Цвет глаз", "Карие");
        break;
    case "Мужчина (высокий рост)":
        traits.put("Рост", 195.0);
        traits.put("Вес", 90.0);
        traits.put("Мускулатура", "Средняя");
        break;
    case "Женщина (светлые волосы)":
        traits.put("Цвет волос", "Светлые");
        traits.put("Цвет глаз", "Голубые");
        traits.put("Рост", 170.0);
        traits.put("Вес", 58.0);
        break;
    case "Случайный основатель (чередование)":
        if (isMale) {
            traits.put("Рост", 170.0 + rand.nextDouble() * 20);
            traits.put("Вес", 70.0 + rand.nextDouble() * 20);
        } else {
            traits.put("Рост", 160.0 + rand.nextDouble() * 15);
            traits.put("Вес", 55.0 + rand.nextDouble() * 15);
        }
        String[] muscles = {"Слабая", "Средняя", "Сильная"};
        traits.put("Мускулатура", muscles[rand.nextInt(muscles.length)]);
```

```
            break;
        }
        engine.addCustomFounder(name, isMale, traits);
    }
    private void addCustomFounder() {
        JDialog dialog = new JDialog((Frame)null, "Добавить кастомного основателя", true);
        dialog.setLayout(new BorderLayout());
        dialog.setSize(500, 650);
        dialog.setLocationRelativeTo(this);
        JTabbedPane tabbedPane = new JTabbedPane();
        JPanel infoPanel = new JPanel(new GridLayout(3, 2, 5, 5));
        JTextField nameField = new JTextField("Основатель");
            JComboBox<String> genderCombo = new JComboBox<>(new String[]{"Мужчина",
"Женщина"});
        infoPanel.add(new JLabel("Имя:"));
        infoPanel.add(nameField);
        infoPanel.add(new JLabel("Пол:"));
        infoPanel.add(genderCombo);
        infoPanel.add(new JLabel(""));
        infoPanel.add(new JLabel(""));
        JPanel physicalPanel = new JPanel(new GridLayout(0, 4, 5, 5));
        physicalPanel.add(new JLabel("Рост (см):"));
        JSlider heightSlider = new JSlider(150, 200, 175);
        heightSlider.setMajorTickSpacing(10);
        heightSlider.setMinorTickSpacing(5);
        heightSlider.setPaintTicks(true);
        heightSlider.setPaintLabels(true);
        JLabel heightValue = new JLabel("175 см");
        heightSlider.addChangeListener(e -> {
            heightValue.setText(heightSlider.getValue() + " см");
        });
        physicalPanel.add(heightSlider);
        physicalPanel.add(new JLabel(""));
        physicalPanel.add(heightValue);
```

```
        physicalPanel.add(new JLabel("Вес (кг):"));
        JSlider weightSlider = new JSlider(40, 120, 70);
        weightSlider.setMajorTickSpacing(20);
        weightSlider.setMinorTickSpacing(5);
        weightSlider.setPaintTicks(true);
        weightSlider.setPaintLabels(true);
        JLabel weightValue = new JLabel("70 кг");
        weightSlider.addChangeListener(e -> {
            weightValue.setText(weightSlider.getValue() + " кг");
        });
        physicalPanel.add(weightSlider);
        physicalPanel.add(new JLabel(""));
        physicalPanel.add(weightValue);
        physicalPanel.add(new JLabel("Мускулатура:"));
                JComboBox<String> muscleCombo = new JComboBox<>(new String[]{"Слабая",
"Средняя", "Сильная"});
        physicalPanel.add(muscleCombo);
        physicalPanel.add(new JLabel(""));
        physicalPanel.add(new JLabel(""));
        physicalPanel.add(new JLabel("Размер ушей:"));
                JComboBox<String> earCombo = new JComboBox<>(new String[]{"Маленькие",
"Средние", "Большие"});
        physicalPanel.add(earCombo);
        physicalPanel.add(new JLabel(""));
        physicalPanel.add(new JLabel(""));
        JLabel bmiLabel = new JLabel("ИМТ: - ( - )");
        bmiLabel.setFont(new Font("Arial", Font.BOLD, 12));
        physicalPanel.add(new JLabel("Индекс массы тела:"));
        physicalPanel.add(bmiLabel);
        physicalPanel.add(new JLabel(""));
        physicalPanel.add(new JLabel(""));
        ChangeListener bmiUpdater = e -> {
            double height = heightSlider.getValue() / 100.0;
            double weight = weightSlider.getValue();
```

```
            double bmi = weight / (height * height);
            String bmiCategory;
            if (bmi < 18.5) bmiCategory = "Недостаточный";
            else if (bmi < 25) bmiCategory = "Нормальный";
            else if (bmi < 30) bmiCategory = "Избыточный";
            else bmiCategory = "Ожирение";
            bmiLabel.setText(String.format("ИМТ: %.1f (%s)", bmi, bmiCategory));
        };
        heightSlider.addChangeListener(bmiUpdater);
        weightSlider.addChangeListener(bmiUpdater);
        bmiUpdater.stateChanged(null);
        JPanel categoricalPanel = new JPanel(new GridLayout(0, 4, 5, 5));
        Map<String, JComboBox<String>> combos = new HashMap<>();
        int count = 0;
        for (String traitName : Human.TRAIT_VALUES.keySet()) {
            if (traitName.equals("Наличие веснушек") || traitName.equals("Телосложение")) {
                continue;
            }
            categoricalPanel.add(new JLabel(traitName + ":"));
                                                    JComboBox<String> combo = new
JComboBox<>(Human.TRAIT_VALUES.get(traitName));
            combos.put(traitName, combo);
            categoricalPanel.add(combo);
            count++;
            if (count % 2 == 0) {
                categoricalPanel.add(new JLabel(""));
                categoricalPanel.add(new JLabel(""));
            }
        }
        categoricalPanel.add(new JLabel("Наличие веснушек:"));
                                            JComboBox<String> frecklesCombo = new
JComboBox<>(Human.TRAIT_VALUES.get("Наличие веснушек"));
        combos.put("Наличие веснушек", frecklesCombo);
        categoricalPanel.add(frecklesCombo);
```

```
categoricalPanel.add(new JLabel(""));
categoricalPanel.add(new JLabel(""));
tabbedPane.addTab("Основное", infoPanel);
tabbedPane.addTab("Физические параметры", new JScrollPane(physicalPanel));
tabbedPane.addTab("Внешность", new JScrollPane(categoricalPanel));
JButton createButton = new JButton("Создать");
createButton.addActionListener(e -> {
    String name = nameField.getText().trim();
    if (name.isEmpty()) {
        JOptionPane.showMessageDialog(dialog, "Введите имя основателя!");
        return;
    }
    boolean isMale = genderCombo.getSelectedIndex() == 0;
    Map<String, Object> traits = new HashMap<>();
    traits.put("Рост", (double) heightSlider.getValue());
    traits.put("Вес", (double) weightSlider.getValue());
    traits.put("Мускулатура", muscleCombo.getSelectedItem());
    traits.put("Размер ушей", earCombo.getSelectedItem());
    for (String traitName : combos.keySet()) {
        traits.put(traitName, combos.get(traitName).getSelectedItem());
    }
    double heightM = heightSlider.getValue() / 100.0;
    double weight = weightSlider.getValue();
    double bmi = weight / (heightM * heightM);
    String bodyType;
    if (bmi < 18.5) {
        bodyType = "Астеническое";
    } else if (bmi < 25) {
        bodyType = "Нормостеническое";
    } else {
        bodyType = "Гиперстеническое";
    }
    traits.put("Телосложение", bodyType);
    engine.addCustomFounder(name, isMale, traits);
```

```
            dialog.dispose();
        });
        JButton cancelButton = new JButton("Отмена");
        cancelButton.addActionListener(e -> dialog.dispose());
        JPanel buttonPanel = new JPanel(new FlowLayout());
        buttonPanel.add(createButton);
        buttonPanel.add(cancelButton);
        dialog.add(tabbedPane, BorderLayout.CENTER);
        dialog.add(buttonPanel, BorderLayout.SOUTH);
        dialog.setVisible(true);
    }
    private void updateStatus() {
        if (engine.isEvolutionRunning()) {
            statusLabel.setText("Эволюция запущена...");
            progressBar.setValue((engine.getCurrentYear() % 100));
        } else {
            statusLabel.setText(String.format(
                    "Готово | Год: %d | Поколений: %d | Живых: %d | Всего: %d | Ср.присп: %.2f",
                    engine.getCurrentYear(), engine.getMaxGeneration(), engine.getAlivePeopleCount(),
                    engine.getTotalPeopleCount(), engine.getAverageFitness()));
            progressBar.setValue(0);
        }
    }
    public boolean isOptimizeRenderEnabled() {
        return optimizeRenderCheck.isSelected();
    }
    public JCheckBox getOptimizeRenderCheck() {
        return optimizeRenderCheck;
    }
}
```

Файл DetailsPanel.java

Код:

```
import javax.swing.*;
```

```
import javax.swing.event.*;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.io.File;
import java.util.ArrayList;
import java.util.Map;
import java.util.TreeMap;
import java.util.List;
import javax.imageio.ImageIO;
class DetailsPanel extends JPanel implements ChangeListener {
    private EvolutionEngine engine;
    private JLabel nameLabel;
    private JLabel infoLabel;
    private JPanel traitsPanel;
    private JLabel imageLabel;
    private BufferedImage maleImage;
    private BufferedImage femaleImage;
    private JTextArea historyArea;
    private Human lastDisplayedHuman = null;
    private JButton showFamilyTreeButton;
    public DetailsPanel(EvolutionEngine engine) {
        this.engine = engine;
        engine.addChangeListener(this);
        setLayout(new BorderLayout(5, 5));
        setBorder(BorderFactory.createTitledBorder("Детали особи"));
        setPreferredSize(new Dimension(400, 800));

        initializeComponents();
        loadImages();
    }
    private void initializeComponents() {
        JPanel topPanel = new JPanel(new BorderLayout());
        nameLabel = new JLabel("Не выбрано", SwingConstants.CENTER);
        nameLabel.setFont(new Font("Arial", Font.BOLD, 18));
```

```
nameLabel.setBorder(BorderFactory.createEmptyBorder(5, 5, 5, 5));
infoLabel = new JLabel(" ", SwingConstants.CENTER);
infoLabel.setFont(new Font("Arial", Font.PLAIN, 12));
topPanel.add(nameLabel, BorderLayout.NORTH);
topPanel.add(infoLabel, BorderLayout.SOUTH);
imageLabel = new JLabel();
imageLabel.setHorizontalAlignment(SwingConstants.CENTER);
imageLabel.setBorder(BorderFactory.createLineBorder(Color.GRAY, 2));
JPanel imagePanel = new JPanel(new BorderLayout());
imagePanel.add(imageLabel, BorderLayout.CENTER);
imagePanel.setPreferredSize(new Dimension(380, 250));
traitsPanel = new JPanel(new GridLayout(0, 2, 5, 5));
traitsPanel.setBorder(BorderFactory.createTitledBorder("Генетические признаки"));
JScrollPane traitsScroll = new JScrollPane(traitsPanel);
traitsScroll.setPreferredSize(new Dimension(380, 200));
historyArea = new JTextArea(8, 30);
historyArea.setEditable(false);
historyArea.setFont(new Font("Monospaced", Font.PLAIN, 11));
historyArea.setLineWrap(true);
historyArea.setWrapStyleWord(true);
JScrollPane historyScroll = new JScrollPane(historyArea);
historyScroll.setBorder(BorderFactory.createTitledBorder("Генеалогия"));
showFamilyTreeButton = new JButton("Показать семейное дерево");
showFamilyTreeButton.setEnabled(false);
showFamilyTreeButton.addActionListener(e -> {
   Human human = engine.getSelectedHuman();
   if (human != null) {
      JFrame parentFrame = (JFrame) SwingUtilities.getWindowAncestor(this);
      FamilyTreeDialog dialog = new FamilyTreeDialog(parentFrame, engine, human);
      dialog.setVisible(true);
   }
});
JPanel buttonPanel = new JPanel(new FlowLayout());
buttonPanel.add(showFamilyTreeButton);
```

```
    JPanel centerPanel = new JPanel(new BorderLayout());
    centerPanel.add(imagePanel, BorderLayout.NORTH);
    centerPanel.add(traitsScroll, BorderLayout.CENTER);
    centerPanel.add(buttonPanel, BorderLayout.SOUTH);
    add(topPanel, BorderLayout.NORTH);
    add(centerPanel, BorderLayout.CENTER);
    add(historyScroll, BorderLayout.SOUTH);
}
private void loadImages() {
    SwingWorker<Void, Void> worker = new SwingWorker<Void, Void>() {
        @Override
        protected Void doInBackground() throws Exception {
            try {
                File maleFile = new File("man.jpeg");
                if (!maleFile.exists()) {
                    maleFile = new File("src/man.jpeg");
                }
                if (!maleFile.exists()) {
                    maleFile = new File("./man.jpeg");
                }
                if (maleFile.exists()) {
                    maleImage = ImageIO.read(maleFile);
                } else {
                    createDefaultMaleImage();
                }
                File femaleFile = new File("woman.jpeg");
                if (!femaleFile.exists()) {
                    femaleFile = new File("src/woman.jpeg");
                }
                if (!femaleFile.exists()) {
                    femaleFile = new File("./woman.jpeg");
                }
                if (femaleFile.exists()) {
                    femaleImage = ImageIO.read(femaleFile);
```

```
            } else {
                createDefaultFemaleImage();
            }
        } catch (Exception e) {
            System.err.println("Ошибка загрузки изображений: " + e.getMessage());
            createDefaultImages();
        }
        return null;
    }
    @Override
    protected void done() {
        updateDisplay();
    }
  };
  worker.execute();
}
private void createDefaultMaleImage() {
  maleImage = new BufferedImage(300, 400, BufferedImage.TYPE_INT_RGB);
  Graphics2D g2d = maleImage.createGraphics();
  GradientPaint gradient = new GradientPaint(0, 0, new Color(173, 216, 230),
      300, 400, new Color(135, 206, 235));
  g2d.setPaint(gradient);
  g2d.fillRect(0, 0, 300, 400);
  g2d.setColor(new Color(100, 150, 200));
  g2d.fillOval(100, 50, 100, 100);
  g2d.fillRect(125, 150, 50, 150);
  g2d.fillRect(75, 150, 50, 20);
  g2d.fillRect(175, 150, 50, 20);
  g2d.fillRect(125, 300, 20, 80);
  g2d.fillRect(155, 300, 20, 80);
  g2d.setColor(Color.BLACK);
  g2d.setFont(new Font("Arial", Font.BOLD, 18));
  g2d.drawString("♂ Мужчина", 90, 200);
  g2d.setFont(new Font("Arial", Font.PLAIN, 12));
```

```
    g2d.drawString("(файл man.jpeg)", 95, 220);
    g2d.drawString("не найден", 110, 240);
    g2d.dispose();
}
private void createDefaultFemaleImage() {
    femaleImage = new BufferedImage(300, 400, BufferedImage.TYPE_INT_RGB);
    Graphics2D g2d = femaleImage.createGraphics();
    GradientPaint gradient = new GradientPaint(0, 0, new Color(255, 182, 193),
            300, 400, new Color(255, 105, 180));
    g2d.setPaint(gradient);
    g2d.fillRect(0, 0, 300, 400);
    g2d.setColor(new Color(200, 120, 160));
    g2d.fillOval(100, 50, 100, 100);
    g2d.fillRect(125, 150, 50, 100);
    int[] xPoints = {100, 200, 175, 125, 100};
    int[] yPoints = {250, 250, 300, 300, 250};
    g2d.fillPolygon(xPoints, yPoints, 5);
    g2d.fillRect(75, 150, 50, 20);
    g2d.fillRect(175, 150, 50, 20);
    g2d.setColor(Color.BLACK);
    g2d.setFont(new Font("Arial", Font.BOLD, 18));
    g2d.drawString("♀ Женщина", 90, 200);
    g2d.setFont(new Font("Arial", Font.PLAIN, 12));
    g2d.drawString("(файл woman.jpeg)", 90, 220);
    g2d.drawString("не найден", 110, 240);
    g2d.dispose();
}
private void createDefaultImages() {
    createDefaultMaleImage();
    createDefaultFemaleImage();
}
private void updateDisplay() {
    Human human = engine.getSelectedHuman();
    if (human == lastDisplayedHuman && human != null) {
```

```
            return;
        }
        lastDisplayedHuman = human;
        if (human == null) {
            nameLabel.setText("Не выбрано");
            infoLabel.setText("Выберите особь на дереве");
            traitsPanel.removeAll();
            historyArea.setText("");
            showFamilyTreeButton.setEnabled(false);
            if (maleImage != null) {
                                Image scaledImage = maleImage.getScaledInstance(350, 230,
Image.SCALE_SMOOTH);
                imageLabel.setIcon(new ImageIcon(scaledImage));
            }
            return;
        }
        nameLabel.setText(human.name);
        double height = human.getHeight();
        double weight = human.getWeight();
        double heightM = height / 100.0;
        double bmi = weight / (heightM * heightM);
                    infoLabel.setText(String.format("<html>%s | ID: %d | Поколение: %d |
%s<br>Приспособленность: %.2f | Рост: %.0f см | Вес: %.1f кг | ИМТ: %.1f</html>",
                human.getGenderText(), human.id, human.generation, human.getStatus(),
                human.fitness, height, weight, bmi));
        SwingUtilities.invokeLater(() -> {
            BufferedImage currentImage = human.isMale ? maleImage : femaleImage;
            if (currentImage != null) {
                                Image scaledImage = currentImage.getScaledInstance(350, 230,
Image.SCALE_SMOOTH);
                imageLabel.setIcon(new ImageIcon(scaledImage));
            }
        });
        SwingUtilities.invokeLater(() -> {
```

```
            traitsPanel.removeAll();
            addTraitCategory("Физические параметры:",
                new String[]{"Рост", "Вес", "Телосложение", "Мускулатура", "Размер ушей"});
            addTraitCategory("Внешность:",
                    new String[]{"Цвет глаз", "Цвет волос", "Цвет кожи", "Форма лица", "Разрез
глаз"});
            addTraitCategory("Детали:",
                new String[]{"Форма носа", "Форма губ", "Тип волос", "Лоб", "Подбородок"});
            addTraitCategory("Особенности:",
                new String[]{"Наличие веснушек"});
            traitsPanel.revalidate();
            traitsPanel.repaint();
            updateHistory(human);
            showFamilyTreeButton.setEnabled(true);
            showFamilyTreeButton.setText("Показать семейное дерево");
        });
    }
    private void addTraitCategory(String categoryName, String[] traitNames) {
        Human human = engine.getSelectedHuman();
        if (human == null) return;
        traitsPanel.add(new JLabel(" "));
        JLabel categoryLabel = new JLabel(categoryName);
        categoryLabel.setFont(new Font("Arial", Font.BOLD, 12));
        categoryLabel.setForeground(new Color(0, 0, 150));
        traitsPanel.add(categoryLabel);
        for (String traitName : traitNames) {
            if (human.traits.containsKey(traitName)) {
                JLabel nameLabel = new JLabel("  " + traitName + ":");
                nameLabel.setFont(new Font("Arial", Font.PLAIN, 11));
                String value = human.getTraitDisplay(traitName);
                JLabel valueLabel = new JLabel(value);
                valueLabel.setFont(new Font("Arial", Font.BOLD, 11));
                if (traitName.equals("Рост") || traitName.equals("Вес")) {
                    valueLabel.setForeground(new Color(0, 100, 0));
```

```
            } else if (traitName.equals("Мускулатура")) {
                if (value.equals("Сильная")) valueLabel.setForeground(new Color(0, 150, 0));
                else if (value.equals("Средняя")) valueLabel.setForeground(new Color(200, 150, 0));
                else valueLabel.setForeground(Color.BLACK);
            } else if (traitName.contains("Цвет")) {
                valueLabel.setForeground(new Color(150, 0, 0));
            } else if (traitName.equals("Телосложение")) {
                 if (value.equals("Нормостеническое")) valueLabel.setForeground(new Color(0, 150,
0));
                else valueLabel.setForeground(Color.BLACK);
            } else {
                valueLabel.setForeground(Color.BLACK);
            }
            traitsPanel.add(nameLabel);
            traitsPanel.add(valueLabel);
        }
    }
  }
  private void updateHistory(Human human) {
    StringBuilder history = new StringBuilder();
    history.append("=== ГЕНЕАЛОГИЧЕСКОЕ ДРЕВО ===\n\n");
    history.append("ПОКОЛЕНИЕ: ").append(human.generation).append("\n");
    history.append("ВОЗРАСТ: ").append(human.age).append(" лет\n");
    history.append("СТАТУС: ").append(human.alive ? "ЖИВ" : "УМЕР").append("\n");
    history.append("\n=== РОДИТЕЛИ ===\n");
    history.append("ОТЕЦ: ");
    if (human.father != null) {
      history.append(human.father.name)
            .append(" (ID: ").append(human.father.id)
            .append(", Gen ").append(human.father.generation)
            .append(", ").append(human.father.age).append(" лет")
            .append(", ").append(human.father.alive ? "жив" : "умер")
            .append(")\n");
    } else {
```

```
    history.append("Неизвестен (основатель)\n");
}
history.append("МАТЬ: ");
if (human.mother != null) {
    history.append(human.mother.name)
            .append(" (ID: ").append(human.mother.id)
            .append(", Gen ").append(human.mother.generation)
            .append(", ").append(human.mother.age).append(" лет")
            .append(", ").append(human.mother.alive ? "жива" : "умерла")
            .append(", супруг: ").append(getSpouseInfo(human.mother))
            .append(")\n");
} else {
    history.append("Неизвестна (основатель)\n");
}
history.append("\n=== ДЕТИ (").append(human.children.size()).append(") ===\n");
if (human.children.isEmpty()) {
    history.append("  Нет детей\n");
} else {
    Map<Integer, List<Human>> childrenByGen = new TreeMap<>();
    for (Human child : human.children) {
        childrenByGen.computeIfAbsent(child.generation, k -> new ArrayList<>()).add(child);
    }
    for (Map.Entry<Integer, List<Human>> entry : childrenByGen.entrySet()) {
        history.append("  Поколение ").append(entry.getKey()).append(":\n");
        for (Human child : entry.getValue()) {
            String fatherInfo = child.father != null ?
                    child.father.name + " (ID:" + child.father.id + ")" : "неизвестен";
            String motherInfo = child.mother != null ?
                    child.mother.name + " (ID:" + child.mother.id + ")" : "неизвестна";
            history.append("    • ").append(child.name)
                    .append(" (ID:").append(child.id)
                    .append(", ").append(child.getGenderSymbol())
                    .append(", ").append(child.age).append(" лет")
                    .append(", ").append(child.alive ? "жив" : "умер")
```

```
                    .append(")\n")
                    .append("       Отец: ").append(fatherInfo)
                    .append(", Мать: ").append(motherInfo).append("\n");
            }
        }
    }
    history.append("\n=== БРАТЬЯ/СЕСТРЫ ===\n");
    if (human.father != null && human.mother != null) {
        List<Human> siblings = new ArrayList<>();
        for (Human child : human.father.children) {
            if (child != human && child.mother == human.mother && !siblings.contains(child)) {
                siblings.add(child);
            }
        }
        if (siblings.isEmpty()) {
            history.append("  Нет братьев/сестер\n");
        } else {
            for (Human sibling : siblings) {
                history.append("  • ").append(sibling.name)
                        .append(" (ID:").append(sibling.id)
                        .append(", ").append(sibling.getGenderSymbol())
                        .append(", Gen ").append(sibling.generation)
                        .append(")\n");
            }
        }
    } else {
        history.append("  Нет информации (основатель)\n");
    }
    historyArea.setText(history.toString());
}
private String getSpouseInfo(Human person) {
    if (person == null) return "нет";
    for (Human child : person.children) {
        if (person.isMale && child.mother != null) {
```

```
                return child.mother.name + " (ID:" + child.mother.id + ")";
            } else if (!person.isMale && child.father != null) {
                return child.father.name + " (ID:" + child.father.id + ")";
            }
        }
        return "нет";
    }
    @Override
    public void stateChanged(ChangeEvent e) {
        SwingUtilities.invokeLater(this::updateDisplay);
    }
}
```

Файл Human.java

Код:

```
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
class Human {
    private static final ThreadLocal<Random> RANDOM = ThreadLocal.withInitial(Random::new);
    final int id;
    final String name;
    final boolean isMale;
    final int generation;
    Human father;
    Human mother;
    final List<Human> children;
    final Map<String, Object> traits;
    int age = 0;
    volatile boolean alive = true;
    int deathAge = -1;
    double fitness = 0;
    public static final Map<String, String[]> TRAIT_VALUES;
    public static final String[] TRAIT_NAMES;
    static {
```

```
        Map<String, String[]> map = new HashMap<>();
        map.put("Цвет глаз", new String[]{"Карие", "Голубые", "Зеленые", "Серые", "Черные"});
            map.put("Цвет волос", new String[]{"Черные", "Каштановые", "Русые", "Рыжие",
"Светлые"});
        map.put("Цвет кожи", new String[]{"Светлая", "Средняя", "Смуглая", "Оливковая"});
                map.put("Телосложение", new String[]{"Астеническое", "Нормостеническое",
"Гиперстеническое"});
        map.put("Тип волос", new String[]{"Прямые", "Волнистые", "Курчавые"});
                 map.put("Форма лица", new String[]{"Овальная", "Круглая", "Квадратная",
"Сердцевидная"});
        map.put("Разрез глаз", new String[]{"Миндалевидный", "Круглый", "Раскосый"});
        map.put("Форма носа", new String[]{"Прямой", "Курносый", "Орлиный", "Широкий"});
        map.put("Форма губ", new String[]{"Тонкие", "Средние", "Пухлые"});
        map.put("Лоб", new String[]{"Высокий", "Средний", "Низкий"});
        map.put("Подбородок", new String[]{"Острый", "Квадратный", "Круглый"});
        map.put("Наличие веснушек", new String[]{"Да", "Нет"});
        TRAIT_VALUES = Collections.unmodifiableMap(map);
        List<String> allTraits = new ArrayList<>(TRAIT_VALUES.keySet());
        allTraits.addAll(Arrays.asList("Рост", "Вес", "Мускулатура", "Размер ушей"));
        TRAIT_NAMES = allTraits.toArray(new String[0]);
    }
    public Human(String name, boolean isMale, int generation,
                 Human father, Human mother, Random rand, int id) {
        this.id = id;
        this.name = name;
        this.isMale = isMale;
        this.generation = generation;
        this.father = father;
        this.mother = mother;
        this.children = Collections.synchronizedList(new ArrayList<>());
        this.traits = new ConcurrentHashMap<>();
        if (father == null || mother == null) {
            initializeRandomTraits(rand);
        } else {
```

```
            inheritTraits(father, mother, rand);
        }
        mutateTraits(rand);
        calculateFitness();
        if (!traits.containsKey("Телосложение")) {
            calculateBodyType();
        }
        if (father != null) father.children.add(this);
        if (mother != null) mother.children.add(this);
    }
    private void initializeRandomTraits(Random rand) {
        double baseHeight = isMale ? 165 : 155;
        double height = baseHeight + rand.nextDouble() * 25;
        traits.put("Рост", height);
        double idealWeight = calculateIdealWeight(height, isMale);
        double weight = idealWeight * (0.85 + rand.nextDouble() * 0.3);
        traits.put("Вес", weight);
        String[] muscleOptions = {"Слабая", "Средняя", "Сильная"};
        traits.put("Мускулатура", muscleOptions[rand.nextInt(muscleOptions.length)]);
        String[] earOptions = {"Маленькие", "Средние", "Большие"};
        traits.put("Размер ушей", earOptions[rand.nextInt(earOptions.length)]);

        for (Map.Entry<String, String[]> entry : TRAIT_VALUES.entrySet()) {
            String[] values = entry.getValue();
            traits.put(entry.getKey(), values[rand.nextInt(values.length)]);
        }
    }
    private double calculateIdealWeight(double height, boolean isMale) {
        return (height - 100) * (isMale ? 1.1 : 1.05);
    }
    private void inheritTraits(Human father, Human mother, Random rand) {
        double fatherHeight = (Double) father.traits.get("Рост");
        double motherHeight = (Double) mother.traits.get("Рост");
        double baseHeight = (fatherHeight + motherHeight) / 2.0;
```

```
baseHeight += isMale ? 5 : -5;
double height = baseHeight * (0.95 + rand.nextDouble() * 0.1);
traits.put("Рост", height);
double idealWeight = calculateIdealWeight(height, isMale);
double fatherWeight = (Double) father.traits.get("Вес");
double motherWeight = (Double) mother.traits.get("Вес");
double weight = idealWeight * 0.7 + ((fatherWeight + motherWeight) / 2.0) * 0.3;
weight *= (0.85 + rand.nextDouble() * 0.3);
traits.put("Вес", weight);
String fatherMuscle = (String) father.traits.get("Мускулатура");
String motherMuscle = (String) mother.traits.get("Мускулатура");
String[] muscleOptions = {"Слабая", "Средняя", "Сильная"};
String inheritedMuscle = rand.nextBoolean() ? fatherMuscle : motherMuscle;
if (rand.nextDouble() < 0.2) {
    int currentIdx = indexOf(muscleOptions, inheritedMuscle);
    if (currentIdx >= 0) {
        if (rand.nextBoolean() && currentIdx < muscleOptions.length - 1) {
            inheritedMuscle = muscleOptions[currentIdx + 1];
        } else if (currentIdx > 0) {
            inheritedMuscle = muscleOptions[currentIdx - 1];
        }
    }
}
traits.put("Мускулатура", inheritedMuscle);
String fatherEars = (String) father.traits.get("Размер ушей");
String motherEars = (String) mother.traits.get("Размер ушей");
String[] earOptions = {"Маленькие", "Средние", "Большие"};
traits.put("Размер ушей", rand.nextBoolean() ? fatherEars : motherEars);
for (String traitName : TRAIT_VALUES.keySet()) {
    if (traitName.equals("Телосложение")) continue;
    String fatherValue = (String) father.traits.get(traitName);
    String motherValue = (String) mother.traits.get(traitName);
    String inheritedValue = rand.nextBoolean() ? fatherValue : motherValue;
    if (isDominantTrait(traitName, fatherValue) && rand.nextDouble() < 0.7) {
```

```
            inheritedValue = fatherValue;
        } else if (isDominantTrait(traitName, motherValue) && rand.nextDouble() < 0.7) {
            inheritedValue = motherValue;
        }
        traits.put(traitName, inheritedValue);
    }
}
private int indexOf(String[] array, String value) {
    for (int i = 0; i < array.length; i++) {
        if (array[i].equals(value)) return i;
    }
    return -1;
}
private boolean isDominantTrait(String traitName, String value) {
    switch (traitName) {
        case "Цвет глаз":
            return value.equals("Карие") || value.equals("Черные");
        case "Цвет волос":
            return value.equals("Черные") || value.equals("Каштановые");
        case "Тип волос":
            return value.equals("Курчавые");
        case "Наличие веснушек":
            return value.equals("Нет");
        default:
            return false;
    }
}
private void mutateTraits(Random rand) {
    if (rand.nextDouble() < 0.05) {
        double currentHeight = (Double) traits.get("Рост");
        traits.put("Рост", Math.max(150, Math.min(200,
                currentHeight + (rand.nextDouble() - 0.5) * 10)));
    }
    if (rand.nextDouble() < 0.05) {
```

```
            double currentWeight = (Double) traits.get("Вес");
            traits.put("Вес", Math.max(40, Math.min(150,
                    currentWeight + (rand.nextDouble() - 0.5) * 10)));
        }
        if (rand.nextDouble() < 0.05) {
            String[] muscleOptions = {"Слабая", "Средняя", "Сильная"};
            String current = (String) traits.get("Мускулатура");
            int idx = indexOf(muscleOptions, current);
            if (idx >= 0) {
                int newIdx;
                do {
                    newIdx = rand.nextInt(muscleOptions.length);
                } while (newIdx == idx);
                traits.put("Мускулатура", muscleOptions[newIdx]);
            }
        }
        for (String traitName : TRAIT_VALUES.keySet()) {
            if (rand.nextDouble() < 0.02) {
                String[] values = TRAIT_VALUES.get(traitName);
                traits.put(traitName, values[rand.nextInt(values.length)]);
            }
        }
    }
    private void calculateBodyType() {
        double height = (Double) traits.get("Рост");
        double weight = (Double) traits.get("Вес");
        double heightM = height / 100.0;
        double bmi = weight / (heightM * heightM);
        String bodyType;
        if (bmi < 18.5) {
            bodyType = "Астеническое";
        } else if (bmi < 25) {
            bodyType = "Нормостеническое";
        } else {
```

```
            bodyType = "Гиперстеническое";
        }
        traits.put("Телосложение", bodyType);
    }
    void calculateFitness() {
        double height = (Double) traits.get("Рост");
        double weight = (Double) traits.get("Вес");
        String muscleStr = (String) traits.get("Мускулатура");
        double musculature;
        switch (muscleStr) {
            case "Сильная": musculature = 0.9; break;
            case "Средняя": musculature = 0.6; break;
            default: musculature = 0.3; break;
        }
        double heightM = height / 100.0;
        double bmi = weight / (heightM * heightM);
        double bmiFitness = 1.0 - Math.abs(bmi - 22) / 10.0;
        double ageFactor = 1.0 - Math.max(0, (age - 30) / 70.0);
        fitness = (bmiFitness * 0.4 + musculature * 0.3 + ageFactor * 0.3);
        fitness = Math.max(0, Math.min(1, fitness));
    }
    public void ageOneYear(Random rand) {
        age++;
        if (!alive) return;
        double baseLongevity = 70 + (fitness * 25);
        double deathChance;
        if (age < 18) {
            deathChance = 0.002;
        } else if (age < 30) {
            deathChance = 0.005;
        } else if (age < baseLongevity) {
            double ageFactor = (age - 30.0) / (baseLongevity - 30.0);
            deathChance = 0.01 + ageFactor * 0.08;
        } else if (age < 100) {
```

```
            double ageFactor = (age - baseLongevity) / (100.0 - baseLongevity);
            deathChance = 0.1 + ageFactor * 0.8;
        } else {
            deathChance = 1.0;
        }
        deathChance *= (1.2 - fitness * 0.4);
        if (age >= 110) {
            alive = false;
            deathAge = age;
            return;
        }
        if (rand.nextDouble() < deathChance) {
            alive = false;
            deathAge = age;
        }
    }
    public boolean canReproduce() {
        return alive && age >= 16 && age <= 50 && fitness > 0.3;
    }
    public String getGenderSymbol() {
        return isMale ? "♂" : "♀";
    }
    public String getGenderText() {
        return isMale ? "Мужчина" : "Женщина";
    }
    public String getStatus() {
        if (alive) {
            return String.format("Жив, %d лет", age);
        } else {
            return String.format("Умер в %d лет", deathAge);
        }
    }
    public String getTraitDisplay(String traitName) {
        Object value = traits.get(traitName);
```

```
        if (traitName.equals("Рост")) {
            return String.format("%.0f см", (Double) value);
        } else if (traitName.equals("Вес")) {
            return String.format("%.1f кг", (Double) value);
        }
        return value.toString();
    }
    @Override
    public String toString() {
        return String.format("%s (ID %d, Gen %d, %s)",
                name, id, generation,
                alive ? age + " лет" : "умер");
    }
    public double getHeight() { return (Double) traits.get("Рост"); }
    public double getWeight() { return (Double) traits.get("Вес"); }
    public String getMuscle() { return (String) traits.get("Мускулатура"); }
    public String getEarSize() { return (String) traits.get("Размер ушей"); }
    public boolean isFounder() {
        return father == null && mother == null && generation == 0;
    }
}
```

Файл FamilyTreeDialog.java

Код:

```
import javax.swing.*;
import javax.swing.event.*;
import java.awt.*;
import java.awt.event.*;
import java.awt.geom.AffineTransform;
import java.awt.image.BufferedImage;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
```

```
import java.util.Map;
import java.util.TreeMap;
import java.util.stream.Collectors;
class FamilyTreeDialog extends JDialog {
    private EvolutionEngine engine;
    private Human founder;
    private FamilyTreePanel treePanel;
    private JLabel statsLabel;
    public FamilyTreeDialog(Frame owner, EvolutionEngine engine, Human founder) {
        super(owner, "Полное семейное дерево: " + founder.name, true);
        this.engine = engine;
        this.founder = founder;
        setLayout(new BorderLayout());
        setSize(1400, 900);
        setLocationRelativeTo(owner);
        initializeComponents();
    }
    private void initializeComponents() {
        JPanel controlPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        controlPanel.setBorder(BorderFactory.createEmptyBorder(5, 5, 5, 5));
        JLabel depthLabel = new JLabel("Глубина дерева:");
        JSlider depthSlider = new JSlider(1, 10, 3);
        depthSlider.setMajorTickSpacing(1);
        depthSlider.setMinorTickSpacing(1);
        depthSlider.setPaintTicks(true);
        depthSlider.setPaintLabels(true);
        depthSlider.setPreferredSize(new Dimension(200, 50));
        JLabel depthValueLabel = new JLabel("3 поколения");
        JCheckBox aliveOnlyCheck = new JCheckBox("Только живые");
        JCheckBox showSpousesCheck = new JCheckBox("Показать супругов", true);
        JButton refreshButton = new JButton("Обновить");
        JButton expandAllButton = new JButton("Показать всё дерево");
        JButton closeButton = new JButton("Закрыть");
        controlPanel.add(depthLabel);
```

```
controlPanel.add(depthSlider);
controlPanel.add(depthValueLabel);
controlPanel.add(Box.createHorizontalStrut(20));
controlPanel.add(aliveOnlyCheck);
controlPanel.add(showSpousesCheck);
controlPanel.add(Box.createHorizontalStrut(20));
controlPanel.add(refreshButton);
controlPanel.add(expandAllButton);
controlPanel.add(closeButton);
statsLabel = new JLabel(" ");
statsLabel.setFont(new Font("Arial", Font.BOLD, 12));
JPanel statsPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
statsPanel.add(statsLabel);
statsPanel.setBorder(BorderFactory.createEmptyBorder(5, 10, 5, 10));
List<Human> initialFamily = engine.getFamilyTree(founder, 3);
treePanel = new FamilyTreePanel(initialFamily, founder);
JScrollPane scrollPane = new JScrollPane(treePanel);
depthSlider.addChangeListener(e -> {
   int depth = depthSlider.getValue();
   depthValueLabel.setText(depth + " поколения");
   updateFamilyTree(depth, aliveOnlyCheck.isSelected(), showSpousesCheck.isSelected());
});
ChangeListener filterListener = e -> {
   int depth = depthSlider.getValue();
   updateFamilyTree(depth, aliveOnlyCheck.isSelected(), showSpousesCheck.isSelected());
};
aliveOnlyCheck.addChangeListener(filterListener);
showSpousesCheck.addChangeListener(filterListener);
refreshButton.addActionListener(e -> {
   int depth = depthSlider.getValue();
   updateFamilyTree(depth, aliveOnlyCheck.isSelected(), showSpousesCheck.isSelected());
});
expandAllButton.addActionListener(e -> {
   List<Human> fullFamily = engine.getAllDescendants(founder);
```

```
        if (aliveOnlyCheck.isSelected()) {
            fullFamily = fullFamily.stream()
                    .filter(h -> h.alive)
                    .collect(Collectors.toList());
        }
        treePanel.updateFamilyMembers(fullFamily);
        updateStatistics(fullFamily);
        depthSlider.setValue(10);
    });
    closeButton.addActionListener(e -> dispose());
    add(controlPanel, BorderLayout.NORTH);
    add(scrollPane, BorderLayout.CENTER);
    add(statsPanel, BorderLayout.SOUTH);
    updateStatistics(initialFamily);
    setDefaultCloseOperation(DISPOSE_ON_CLOSE);
    addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });
}
private void updateFamilyTree(int depth, boolean aliveOnly, boolean showSpouses) {
    List<Human> family = engine.getFamilyTree(founder, depth);
    if (aliveOnly) {
        family = family.stream()
                .filter(h -> h.alive)
                .collect(Collectors.toList());
    }
    if (!showSpouses) {
        List<Human> filtered = new ArrayList<>();
        for (Human person : family) {
            if (isAncestorOrDescendant(person, founder) || person == founder) {
                filtered.add(person);
```

```
            }
        }
        family = filtered;
    }
    treePanel.updateFamilyMembers(family);
    updateStatistics(family);
}
private boolean isAncestorOrDescendant(Human person, Human root) {
    if (person == null || root == null) return false;
    Human current = root;
    while (current != null) {
        if (current == person) return true;
        if (current.father == person || current.mother == person) return true;
        current = current.father;
    }
    return checkDescendants(root, person);
}
private boolean checkDescendants(Human root, Human target) {
    if (root == null) return false;
    if (root == target) return true;
    for (Human child : root.children) {
        if (checkDescendants(child, target)) {
            return true;
        }
    }

    return false;
}
private void updateStatistics(List<Human> family) {
    if (family.isEmpty()) {
        statsLabel.setText("Нет членов семьи");
        return;
    }
    long aliveCount = family.stream().filter(h -> h.alive).count();
```

```
        long maleCount = family.stream().filter(h -> h.isMale).count();
        long femaleCount = family.size() - maleCount;
        int maxGeneration = family.stream().mapToInt(h -> h.generation).max().orElse(0);
        statsLabel.setText(String.format(
                "<html><b>Статистика семьи:</b> Всего членов: %d | Живых: %d | Мужчин: %d |
Женщин: %d | Поколений: %d | Основатель: %s (ID: %d)</html>",
                family.size(), aliveCount, maleCount, femaleCount, maxGeneration + 1, founder.name,
founder.id
        ));
    }
}
class FamilyTreePanel extends JPanel {
    private List<Human> familyMembers;
    private Human rootPerson;
    private Map<Human, Rectangle> nodeBounds = new HashMap<>();
    private Map<Human, Point> nodePositions = new HashMap<>();
    private double zoom = 1.0;
    private Point offset = new Point(50, 50);
    private final int NODE_WIDTH = 160;
    private final int NODE_HEIGHT = 70;
    private final int VERT_SPACING = 120;
    private final int HORIZ_SPACING = 60;
    private Point dragStart;
    private Point lastMousePos;
    private boolean isDragging = false;
    private boolean isRightDragging = false;
    private double zoomStep = 0.05;
    private double minZoom = 0.1;
    private double maxZoom = 5.0;
    private BufferedImage bufferImage = null;
    private boolean bufferDirty = true;
    public FamilyTreePanel(List<Human> familyMembers, Human rootPerson) {
        this.familyMembers = familyMembers;
        this.rootPerson = rootPerson;
```

```
setBackground(new Color(248, 248, 255));
setPreferredSize(new Dimension(3000, 2000));
addMouseListener(new MouseAdapter() {
    public void mousePressed(MouseEvent e) {
        if (SwingUtilities.isLeftMouseButton(e)) {
            dragStart = e.getPoint();
            lastMousePos = e.getPoint();
            isDragging = true;
            setCursor(new Cursor(Cursor.MOVE_CURSOR));
        } else if (SwingUtilities.isRightMouseButton(e)) {
            dragStart = e.getPoint();
            lastMousePos = e.getPoint();
            isRightDragging = true;
            setCursor(new Cursor(Cursor.N_RESIZE_CURSOR));
        }
    }
    public void mouseReleased(MouseEvent e) {
        if (SwingUtilities.isLeftMouseButton(e)) {
            isDragging = false;
            setCursor(new Cursor(Cursor.DEFAULT_CURSOR));
        } else if (SwingUtilities.isRightMouseButton(e)) {
            isRightDragging = false;
            setCursor(new Cursor(Cursor.DEFAULT_CURSOR));
        }
    }
    public void mouseClicked(MouseEvent e) {
        if (SwingUtilities.isLeftMouseButton(e) && e.getClickCount() == 2) {
            double oldZoom = zoom;
            zoom = Math.min(maxZoom, Math.max(minZoom, zoom * 1.5));
            Point mousePos = e.getPoint();
            offset.x = (int)(mousePos.x - (mousePos.x - offset.x) * (zoom / oldZoom));
            offset.y = (int)(mousePos.y - (mousePos.y - offset.y) * (zoom / oldZoom));
            bufferDirty = true;
            repaint();
```

```
        }
    }
});
addMouseMotionListener(new MouseMotionAdapter() {
    public void mouseDragged(MouseEvent e) {
        if (isDragging && dragStart != null) {
            int dx = e.getX() - lastMousePos.x;
            int dy = e.getY() - lastMousePos.y;
            offset.x += dx;
            offset.y += dy;
            lastMousePos = e.getPoint();
            bufferDirty = true;
            repaint();
        } else if (isRightDragging && dragStart != null) {
            int dy = lastMousePos.y - e.getY();
            double scaleFactor = 1.0 + (dy * 0.005);
            double oldZoom = zoom;
            zoom = Math.min(maxZoom, Math.max(minZoom, zoom * scaleFactor));
            if (oldZoom != zoom) {
                offset.x = (int)(dragStart.x - (dragStart.x - offset.x) * (zoom / oldZoom));
                offset.y = (int)(dragStart.y - (dragStart.y - offset.y) * (zoom / oldZoom));
                bufferDirty = true;
            }
            lastMousePos = e.getPoint();
            repaint();
        }
    }
});
addMouseWheelListener(new MouseWheelListener() {
    @Override
    public void mouseWheelMoved(MouseWheelEvent e) {
        double oldZoom = zoom;
        Point mousePos = e.getPoint();
        int notches = e.getWheelRotation();
```

```
            if (notches < 0) {
                zoom = Math.min(maxZoom, zoom * (1.0 + zoomStep));
            } else {
                zoom = Math.max(minZoom, zoom * (1.0 - zoomStep));
            }
            if (oldZoom != zoom) {
                offset.x = (int)(mousePos.x - (mousePos.x - offset.x) * (zoom / oldZoom));
                offset.y = (int)(mousePos.y - (mousePos.y - offset.y) * (zoom / oldZoom));
                bufferDirty = true;
            }
            repaint();
        }
    });
}
public void updateFamilyMembers(List<Human> newFamilyMembers) {
    this.familyMembers = newFamilyMembers;
    bufferDirty = true;
    repaint();
}
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);

    if (bufferImage == null || bufferDirty) {
        createBufferImage();
    }
    g.drawImage(bufferImage, 0, 0, null);
}
private void createBufferImage() {
                        bufferImage = new BufferedImage(getWidth(), getHeight(),
BufferedImage.TYPE_INT_ARGB);
    Graphics2D g2d = bufferImage.createGraphics();
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);
```

```
        g2d.setRenderingHint(RenderingHints.KEY_RENDERING,
                RenderingHints.VALUE_RENDER_QUALITY);
        g2d.translate(offset.x, offset.y);
        g2d.scale(zoom, zoom);
        nodeBounds.clear();
        nodePositions.clear();
        layoutTree();
        drawConnections(g2d);
        drawNodes(g2d);
        g2d.setTransform(new AffineTransform());
        g2d.setColor(Color.BLACK);
        g2d.setFont(new Font("Arial", Font.BOLD, 14));
        g2d.drawString(String.format("Семейное дерево: %s (ID: %d)",
                rootPerson.name, rootPerson.id), 20, 30);
        g2d.setFont(new Font("Arial", Font.PLAIN, 12));
          g2d.drawString(String.format("Масштаб: %.1fx | Всего членов: %d | Управление: ЛКМ-
перемещение, ПКМ-масштаб, Колёсико-зум",
                zoom, familyMembers.size()), 20, 50);
        g2d.dispose();
        bufferDirty = false;
    }
    private void layoutTree() {
        if (familyMembers.isEmpty()) return;
        Map<Integer, List<Human>> byGeneration = new TreeMap<>();
        for (Human human : familyMembers) {
            byGeneration.computeIfAbsent(human.generation,
                    k -> new ArrayList<>()).add(human);
        }
        int startY = 100;
        int maxNodesInGeneration = 1;
        for (List<Human> generation : byGeneration.values()) {
            maxNodesInGeneration = Math.max(maxNodesInGeneration, generation.size());
        }
```

```
            int dynamicHorizSpacing = Math.max(HORIZ_SPACING, 800 / Math.max(1, maxNodesInGeneration));
        for (Map.Entry<Integer, List<Human>> entry : byGeneration.entrySet()) {
            int generation = entry.getKey();
            List<Human> generationPeople = entry.getValue();
            generationPeople.sort(Comparator.comparingInt(h -> h.id));
            int y = startY + generation * VERT_SPACING;
             int totalWidth = generationPeople.size() * (NODE_WIDTH + dynamicHorizSpacing) - dynamicHorizSpacing;
            int startX = 200;
            for (int i = 0; i < generationPeople.size(); i++) {
                Human human = generationPeople.get(i);
                int x = startX + i * (NODE_WIDTH + dynamicHorizSpacing);
                nodeBounds.put(human, new Rectangle(x, y, NODE_WIDTH, NODE_HEIGHT));
                nodePositions.put(human, new Point(x + NODE_WIDTH/2, y + NODE_HEIGHT/2));
            }
        }
    }
    private void drawConnections(Graphics2D g2d) {
        for (Human human : familyMembers) {
            if (human.father != null && familyMembers.contains(human.father)) {
                drawConnection(g2d, human.father, human, new Color(0, 0, 255, 180));
            }
            if (human.mother != null && familyMembers.contains(human.mother)) {
                drawConnection(g2d, human.mother, human, new Color(255, 0, 0, 180));
            }
        }
    }
    private void drawConnection(Graphics2D g2d, Human parent, Human child, Color color) {
        Point parentPos = nodePositions.get(parent);
        Point childPos = nodePositions.get(child);
        if (parentPos == null || childPos == null) return;
        int startX = parentPos.x;
        int startY = parentPos.y + NODE_HEIGHT/2;
```

```
        int endX = childPos.x;
        int endY = childPos.y - NODE_HEIGHT/2;
        g2d.setColor(color);
                                g2d.setStroke(new BasicStroke(2.0f, BasicStroke.CAP_ROUND,
BasicStroke.JOIN_ROUND));
        int ctrlY1 = startY + Math.abs(endY - startY) / 3;
        int ctrlY2 = endY - Math.abs(endY - startY) / 3;
        g2d.drawLine(startX, startY, startX, ctrlY1);
        g2d.drawLine(startX, ctrlY1, endX, ctrlY2);
        g2d.drawLine(endX, ctrlY2, endX, endY);
        drawArrow(g2d, endX, endY, startX, startY, color);
    }
    private void drawArrow(Graphics2D g2d, int x, int y, int fromX, int fromY, Color color) {
        int arrowSize = 8;
        double angle = Math.atan2(y - fromY, x - fromX);
        int x1 = (int)(x - arrowSize * Math.cos(angle - Math.PI / 6));
        int y1 = (int)(y - arrowSize * Math.sin(angle - Math.PI / 6));
        int x2 = (int)(x - arrowSize * Math.cos(angle + Math.PI / 6));
        int y2 = (int)(y - arrowSize * Math.sin(angle + Math.PI / 6));
        g2d.setColor(color);
        g2d.fillPolygon(new int[]{x, x1, x2}, new int[]{y, y1, y2}, 3);
        g2d.setColor(Color.BLACK);
        g2d.setStroke(new BasicStroke(0.8f));
        g2d.drawPolygon(new int[]{x, x1, x2}, new int[]{y, y1, y2}, 3);
    }
    private void drawNodes(Graphics2D g2d) {
        for (Map.Entry<Human, Rectangle> entry : nodeBounds.entrySet()) {
            Human human = entry.getKey();
            Rectangle rect = entry.getValue();
            drawHumanNode(g2d, human, rect.x, rect.y);
        }
    }
    private void drawHumanNode(Graphics2D g2d, Human human, int x, int y) {
        Color bgColor;
```

```
if (!human.alive) {
    bgColor = new Color(200, 200, 200, 220);
} else if (human == rootPerson) {
    bgColor = new Color(255, 255, 150, 240);
} else if (human.isMale) {
    bgColor = new Color(173, 216, 230, 240);
} else {
    bgColor = new Color(255, 182, 193, 240);
}
g2d.setColor(new Color(100, 100, 100, 80));
g2d.fillRoundRect(x + 3, y + 3, NODE_WIDTH, NODE_HEIGHT, 15, 15);
g2d.setColor(bgColor);
g2d.fillRoundRect(x, y, NODE_WIDTH, NODE_HEIGHT, 15, 15);
g2d.setColor(human == rootPerson ? Color.RED :
        (human.isFounder() ? new Color(0, 100, 0) : Color.BLACK));
g2d.setStroke(new BasicStroke(human == rootPerson ? 3.0f :
        (human.isFounder() ? 2.5f : 2.0f)));
g2d.drawRoundRect(x, y, NODE_WIDTH, NODE_HEIGHT, 15, 15);
g2d.setFont(new Font("Arial", Font.PLAIN, 11));
String displayName = human.name;
if (displayName.length() > 18) {
    displayName = displayName.substring(0, 16) + "...";
}
g2d.drawString(displayName, x + 8, y + 18);
String info = human.getGenderSymbol();
if (human.alive) {
    info += " " + human.age + " лет";
} else {
    info += " ✝️ " + (human.deathAge == -1 ? "?" : human.deathAge) + " лет";
}
g2d.drawString(info, x + 8, y + 35);
g2d.setColor(Color.DARK_GRAY);
g2d.drawString("ID:" + human.id + " Gen:" + human.generation, x + 8, y + 52);
int fitnessBar = (int)(human.fitness * (NODE_WIDTH - 20));
```

```
        if (human.fitness > 0.7) {
            g2d.setColor(new Color(0, 180, 0));
        } else if (human.fitness > 0.4) {
            g2d.setColor(new Color(255, 165, 0));
        } else {
            g2d.setColor(Color.RED);
        }
        g2d.fillRect(x + 8, y + NODE_HEIGHT - 10, fitnessBar, 6);
        g2d.setColor(Color.BLACK);
        g2d.setStroke(new BasicStroke(1.0f));
        g2d.drawRect(x + 8, y + NODE_HEIGHT - 10, NODE_WIDTH - 20, 6);
        g2d.setColor(Color.BLACK);
        g2d.setFont(new Font("Arial", Font.BOLD, 10));
        g2d.drawString(String.format("%.0f%%", human.fitness * 100),
                x + NODE_WIDTH - 30, y + NODE_HEIGHT - 4);
    }
}
```

Файл GenealogyTreePanel.java

Код:

```
import javax.swing.JPanel;
import javax.swing.event.ChangeEvent;
import javax.swing.event.ChangeListener;
import java.awt.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.awt.event.MouseMotionAdapter;
import java.awt.event.MouseWheelEvent;
import java.awt.event.MouseWheelListener;
import java.awt.geom.AffineTransform;
import java.awt.geom.CubicCurve2D;
import java.awt.image.BufferedImage;
import java.util.ArrayList;
import java.util.Comparator;
```

```
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import javax.swing.*;
class GenealogyTreePanel extends JPanel implements ChangeListener {
    private EvolutionEngine engine;
    private double zoom = 1.0;
    private Point offset = new Point(50, 50);
    private Point dragStart;
    private Point lastMousePos;
    private final int NODE_WIDTH = 160;
    private final int NODE_HEIGHT = 70;
    private final int VERT_SPACING = 120;
    private final int HORIZ_SPACING = 60;
    private Map<Human, Rectangle> nodeBounds = new HashMap<>();
    private Map<Human, Point> nodeCenters = new HashMap<>();
    private JButton zoomInButton;
    private JButton zoomOutButton;
    private JButton resetZoomButton;
    private JButton fitToScreenButton;
    private boolean optimizeRendering = true;
    private int lastRepaintCount = 0;
    private long lastRepaintTime = 0;
    private BufferedImage bufferImage = null;
    private boolean bufferDirty = true;
    private final Object bufferLock = new Object();
    private boolean isDragging = false;
    private boolean isRightDragging = false;
    private double zoomStep = 0.05;
    private double minZoom = 0.1;
    private double maxZoom = 5.0;
    public GenealogyTreePanel(EvolutionEngine engine) {
        this.engine = engine;
```

```
engine.addChangeListener(this);
setBackground(new Color(248, 248, 255));
setPreferredSize(new Dimension(3000, 2500));
setLayout(null);
createControlPanel();
addMouseListener(new MouseAdapter() {
    public void mousePressed(MouseEvent e) {
        if (SwingUtilities.isLeftMouseButton(e)) {
            dragStart = e.getPoint();
            lastMousePos = e.getPoint();
            isDragging = true;

            if (!checkNodeClick(e.getX(), e.getY())) {
                setCursor(new Cursor(Cursor.MOVE_CURSOR));
            }
        } else if (SwingUtilities.isRightMouseButton(e)) {
            dragStart = e.getPoint();
            lastMousePos = e.getPoint();
            isRightDragging = true;
            setCursor(new Cursor(Cursor.N_RESIZE_CURSOR));
        }
    }
    public void mouseReleased(MouseEvent e) {
        if (SwingUtilities.isLeftMouseButton(e)) {
            dragStart = null;
            isDragging = false;
            setCursor(new Cursor(Cursor.DEFAULT_CURSOR));
            markBufferDirty();
        } else if (SwingUtilities.isRightMouseButton(e)) {
            dragStart = null;
            isRightDragging = false;
            setCursor(new Cursor(Cursor.DEFAULT_CURSOR));
            markBufferDirty();
        }
```

```
    }
    public void mouseClicked(MouseEvent e) {
        if (SwingUtilities.isLeftMouseButton(e) && e.getClickCount() == 2) {
            double oldZoom = zoom;
            zoom = Math.min(maxZoom, Math.max(minZoom, zoom * 1.5));
            Point mousePos = e.getPoint();
            offset.x = (int)(mousePos.x - (mousePos.x - offset.x) * (zoom / oldZoom));
            offset.y = (int)(mousePos.y - (mousePos.y - offset.y) * (zoom / oldZoom));
            markBufferDirty();
            repaint();
        }
    }
});
addMouseMotionListener(new MouseMotionAdapter() {
    public void mouseDragged(MouseEvent e) {
        if (isDragging && dragStart != null) {
            int dx = e.getX() - lastMousePos.x;
            int dy = e.getY() - lastMousePos.y;
            offset.x += dx;
            offset.y += dy;
            lastMousePos = e.getPoint();
            if (Math.abs(dx) > 2 || Math.abs(dy) > 2) {
                optimizedRepaint();
            }
        } else if (isRightDragging && dragStart != null) {
            int dy = lastMousePos.y - e.getY();
            double scaleFactor = 1.0 + (dy * 0.005);
            double oldZoom = zoom;
            zoom = Math.min(maxZoom, Math.max(minZoom, zoom * scaleFactor));
            if (oldZoom != zoom) {
                offset.x = (int)(dragStart.x - (dragStart.x - offset.x) * (zoom / oldZoom));
                offset.y = (int)(dragStart.y - (dragStart.y - offset.y) * (zoom / oldZoom));
                markBufferDirty();
            }
```

```
                lastMousePos = e.getPoint();
                repaint();
            }
        }
    });
    addMouseWheelListener(new MouseWheelListener() {
        @Override
        public void mouseWheelMoved(MouseWheelEvent e) {
            double oldZoom = zoom;
            Point mousePos = e.getPoint();
            int notches = e.getWheelRotation();
            if (notches < 0) {
                zoom = Math.min(maxZoom, zoom * (1.0 + zoomStep));
            } else {
                zoom = Math.max(minZoom, zoom * (1.0 - zoomStep));
            }
            if (oldZoom != zoom) {
                offset.x = (int)(mousePos.x - (mousePos.x - offset.x) * (zoom / oldZoom));
                offset.y = (int)(mousePos.y - (mousePos.y - offset.y) * (zoom / oldZoom));
                markBufferDirty();
            }
            repaint();
        }
    });
}
private void createControlPanel() {
    JPanel controlPanel = new JPanel();
    controlPanel.setLayout(new BoxLayout(controlPanel, BoxLayout.Y_AXIS));
    controlPanel.setOpaque(false);
    controlPanel.setBounds(10, 10, 180, 160);
    Font buttonFont = new Font("Arial", Font.BOLD, 16);
    Font smallFont = new Font("Arial", Font.PLAIN, 12);
    zoomInButton = new JButton("+ Увеличить");
    zoomInButton.setFont(smallFont);
```

```
zoomInButton.setFocusPainted(false);
zoomInButton.setBackground(new Color(220, 240, 255));
zoomInButton.setBorder(BorderFactory.createRaisedBevelBorder());
zoomInButton.setMaximumSize(new Dimension(180, 40));
zoomInButton.addActionListener(e -> {
    double oldZoom = zoom;
    zoom = Math.min(maxZoom, zoom * (1.0 + zoomStep));
    Point center = new Point(getWidth()/2, getHeight()/2);
    offset.x = (int)(center.x - (center.x - offset.x) * (zoom / oldZoom));
    offset.y = (int)(center.y - (center.y - offset.y) * (zoom / oldZoom));
    markBufferDirty();
    repaint();
});
zoomOutButton = new JButton("- Уменьшить");
zoomOutButton.setFont(smallFont);
zoomOutButton.setFocusPainted(false);
zoomOutButton.setBackground(new Color(220, 240, 255));
zoomOutButton.setBorder(BorderFactory.createRaisedBevelBorder());
zoomOutButton.setMaximumSize(new Dimension(180, 40));
zoomOutButton.addActionListener(e -> {
    double oldZoom = zoom;
    zoom = Math.max(minZoom, zoom * (1.0 - zoomStep));
    Point center = new Point(getWidth()/2, getHeight()/2);
    offset.x = (int)(center.x - (center.x - offset.x) * (zoom / oldZoom));
    offset.y = (int)(center.y - (center.y - offset.y) * (zoom / oldZoom));
    markBufferDirty();
    repaint();
});
resetZoomButton = new JButton("1:1 Сбросить масштаб");
resetZoomButton.setFont(smallFont);
resetZoomButton.setFocusPainted(false);
resetZoomButton.setBackground(new Color(255, 240, 220));
resetZoomButton.setBorder(BorderFactory.createRaisedBevelBorder());
resetZoomButton.setMaximumSize(new Dimension(180, 40));
```

```
    resetZoomButton.addActionListener(e -> {
        zoom = 1.0;
        offset.x = 50;
        offset.y = 50;
        markBufferDirty();
        repaint();
    });
    fitToScreenButton = new JButton("⛶ Подогнать под экран");
    fitToScreenButton.setFont(smallFont);
    fitToScreenButton.setFocusPainted(false);
    fitToScreenButton.setBackground(new Color(220, 255, 220));
    fitToScreenButton.setBorder(BorderFactory.createRaisedBevelBorder());
    fitToScreenButton.setMaximumSize(new Dimension(180, 40));
    fitToScreenButton.addActionListener(e -> {
        fitToScreen();
        markBufferDirty();
    });
    controlPanel.add(zoomInButton);
    controlPanel.add(Box.createVerticalStrut(5));
    controlPanel.add(zoomOutButton);
    controlPanel.add(Box.createVerticalStrut(5));
    controlPanel.add(resetZoomButton);
    controlPanel.add(Box.createVerticalStrut(5));
    controlPanel.add(fitToScreenButton);
    add(controlPanel);
}
private boolean checkNodeClick(int x, int y) {
    if (zoomInButton.getBounds().contains(x, y) ||
            zoomOutButton.getBounds().contains(x, y) ||
            resetZoomButton.getBounds().contains(x, y) ||
            fitToScreenButton.getBounds().contains(x, y)) {
        return true;
    }
    for (Map.Entry<Human, Rectangle> entry : nodeBounds.entrySet()) {
```

```
            Rectangle rect = entry.getValue();
            int nodeX = (int)(rect.x * zoom + offset.x);
            int nodeY = (int)(rect.y * zoom + offset.y);
            int nodeWidth = (int)(rect.width * zoom);
            int nodeHeight = (int)(rect.height * zoom);
            if (x >= nodeX && x <= nodeX + nodeWidth &&
                    y >= nodeY && y <= nodeY + nodeHeight) {
                engine.selectHuman(entry.getKey());
                markBufferDirty();
                repaint();
                return true;
            }
        }
        return false;
    }
    private void markBufferDirty() {
        synchronized (bufferLock) {
            bufferDirty = true;
        }
    }
    private void fitToScreen() {
        List<Human> allPeople = engine.getAllPeople();
        if (allPeople.isEmpty()) return;
        int minX = Integer.MAX_VALUE;
        int maxX = Integer.MIN_VALUE;
        int minY = Integer.MAX_VALUE;
        int maxY = Integer.MIN_VALUE;
        Map<Integer, List<Human>> byGeneration = new TreeMap<>();
        for (Human human : allPeople) {
            byGeneration.computeIfAbsent(human.generation,
                    k -> new ArrayList<>()).add(human);
        }
        int startY = 80;
        for (Map.Entry<Integer, List<Human>> entry : byGeneration.entrySet()) {
```

```
            int generation = entry.getKey();
            List<Human> generationPeople = entry.getValue();
            int y = startY + generation * VERT_SPACING;
            int startX = 150;
            for (int i = 0; i < generationPeople.size(); i++) {
                int x = startX + i * (NODE_WIDTH + HORIZ_SPACING);
                minX = Math.min(minX, x);
                maxX = Math.max(maxX, x + NODE_WIDTH);
                minY = Math.min(minY, y);
                maxY = Math.max(maxY, y + NODE_HEIGHT);
            }
        }
        int padding = 100;
        minX -= padding;
        maxX += padding;
        minY -= padding;
        maxY += padding;
        int treeWidth = maxX - minX;
        int treeHeight = maxY - minY;
        int panelWidth = getWidth();
        int panelHeight = getHeight();
        double scaleX = (double) panelWidth / treeWidth;
        double scaleY = (double) panelHeight / treeHeight;
        zoom = Math.min(scaleX, scaleY) * 0.9;
        offset.x = (int) ((panelWidth - treeWidth * zoom) / 2 - minX * zoom);
        offset.y = (int) ((panelHeight - treeHeight * zoom) / 2 - minY * zoom);
        markBufferDirty();
        repaint();
    }
    @Override
    protected void paintComponent(Graphics g) {
        long currentTime = System.currentTimeMillis();
        if (optimizeRendering && currentTime - lastRepaintTime < 16) {
            return;
```

```
        }
        super.paintComponent(g);
        lastRepaintTime = currentTime;
        synchronized (bufferLock) {
          if (bufferImage == null ||
              bufferImage.getWidth() != getWidth() ||
              bufferImage.getHeight() != getHeight() ||
              bufferDirty) {
            if (bufferImage != null) {
              bufferImage.flush();
            }
                                    bufferImage = new BufferedImage(getWidth(), getHeight(),
BufferedImage.TYPE_INT_ARGB);
            Graphics2D bufferGraphics = bufferImage.createGraphics();
            renderToGraphics(bufferGraphics);
            bufferGraphics.dispose();
            bufferDirty = false;
          }
          g.drawImage(bufferImage, 0, 0, null);
        }
        paintChildren(g);
      }
      private void renderToGraphics(Graphics g) {
        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);
        g2d.setRenderingHint(RenderingHints.KEY_RENDERING,
            RenderingHints.VALUE_RENDER_QUALITY);
        g2d.translate(offset.x, offset.y);
        g2d.scale(zoom, zoom);
        nodeBounds.clear();
        nodeCenters.clear();
        drawNodes(g2d);
        drawConnections(g2d);
```

```
        g2d.setTransform(new AffineTransform());
        g2d.setColor(Color.BLACK);
        g2d.setFont(new Font("Arial", Font.PLAIN, 12));
            String info = String.format("Масштаб: %.2fx | Положение: (%d, %d) | Год: %d | Всего особей: %d",
                zoom, offset.x, offset.y, engine.getCurrentYear(), engine.getTotalPeopleCount());
        g2d.drawString(info, 60, 40);
        g2d.setColor(new Color(0, 0, 0, 150));
        g2d.setFont(new Font("Arial", Font.PLAIN, 10));
            String controls = "Управление: ЛКМ - перетаскивание | ПКМ - масштабирование | Колёсико - масштабирование | Двойной клик - приближение";
        g2d.drawString(controls, 60, 60);
    }
    private void drawConnections(Graphics2D g2d) {
        List<Human> allPeople = engine.getAllPeople();
        List<Connection> connections = new ArrayList<>();
        for (Human human : allPeople) {
            if (human.father != null) {
                connections.add(new Connection(human.father, human, new Color(0, 100, 200, 200), "father"));
            }
            if (human.mother != null) {
                connections.add(new Connection(human.mother, human, new Color(200, 50, 50, 200), "mother"));
            }
        }
        connections.sort((c1, c2) -> Integer.compare(c1.parent.generation, c2.parent.generation));
        for (Connection conn : connections) {
            Point parentCenter = nodeCenters.get(conn.parent);
            Point childCenter = nodeCenters.get(conn.child);
            if (parentCenter != null && childCenter != null) {
                drawConnectionLine(g2d, parentCenter, childCenter, conn.color, conn.type);
            }
        }
```

```
    }
     private void drawConnectionLine(Graphics2D g2d, Point from, Point to, Color color, String type) {
        g2d.setColor(color);
        if (type.equals("father")) {
                                    g2d.setStroke(new BasicStroke(2.0f, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));
        } else {
                                    g2d.setStroke(new BasicStroke(2.0f, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND,
                0, new float[]{6, 4}, 0));
        }
        int startX = from.x;
        int startY = from.y + NODE_HEIGHT/2 + 10;
        int endX = to.x;
        int endY = to.y - 10;
        int controlY1 = startY + Math.abs(endY - startY) / 3;
        int controlY2 = endY - Math.abs(endY - startY) / 3;
        if (Math.abs(startY - endY) < 50) {
            controlY1 = startY + 60;
            controlY2 = endY - 60;
        }
        CubicCurve2D curve = new CubicCurve2D.Float(
                startX, startY,
                startX, controlY1,
                endX, controlY2,
                endX, endY
        );
        g2d.draw(curve);
        drawArrow(g2d, endX, endY, startX, startY, color);
        drawParentMarker(g2d, startX, startY, type);
    }
    private void drawArrow(Graphics2D g2d, int x, int y, int fromX, int fromY, Color color) {
        int arrowSize = 10;
```

```
    double dx = x - fromX;
    double dy = y - fromY;
    double angle = Math.atan2(dy, dx);
    int x1 = (int)(x - arrowSize * Math.cos(angle - Math.PI/6));
    int y1 = (int)(y - arrowSize * Math.sin(angle - Math.PI/6));
    int x2 = (int)(x - arrowSize * Math.cos(angle + Math.PI/6));
    int y2 = (int)(y - arrowSize * Math.sin(angle + Math.PI/6));
    g2d.setColor(new Color(color.getRed(), color.getGreen(), color.getBlue(), 255));
    int[] xPoints = {x, x1, x2};
    int[] yPoints = {y, y1, y2};
    g2d.fillPolygon(xPoints, yPoints, 3);
    g2d.setColor(Color.BLACK);
    g2d.setStroke(new BasicStroke(0.8f));
    g2d.drawPolygon(xPoints, yPoints, 3);
}
private void drawParentMarker(Graphics2D g2d, int x, int y, String type) {
    int markerSize = 6;
    g2d.setColor(type.equals("father") ? Color.BLUE : Color.RED);
    g2d.fillOval(x - markerSize/2, y - markerSize/2, markerSize, markerSize);
    g2d.setColor(Color.BLACK);
    g2d.setStroke(new BasicStroke(1.0f));
    g2d.drawOval(x - markerSize/2, y - markerSize/2, markerSize, markerSize);
}
private void drawNodes(Graphics2D g2d) {
    List<Human> allPeople = engine.getAllPeople();
    Map<Integer, List<Human>> byGeneration = new TreeMap<>();
    for (Human human : allPeople) {
        byGeneration.computeIfAbsent(human.generation,
                k -> new ArrayList<>()).add(human);
    }
    int startY = 100;
    for (Map.Entry<Integer, List<Human>> entry : byGeneration.entrySet()) {
        int generation = entry.getKey();
        List<Human> generationPeople = entry.getValue();
```

```
        generationPeople.sort(Comparator.comparingInt(h -> h.id));
        int y = startY + generation * VERT_SPACING;
        int totalWidth = generationPeople.size() * (NODE_WIDTH + HORIZ_SPACING);
        int startX = 180;
        for (int i = 0; i < generationPeople.size(); i++) {
            Human human = generationPeople.get(i);
            int x = startX + i * (NODE_WIDTH + HORIZ_SPACING);
            drawHumanNode(g2d, human, x, y);
            nodeBounds.put(human, new Rectangle(x, y, NODE_WIDTH, NODE_HEIGHT));
            nodeCenters.put(human, new Point(x + NODE_WIDTH/2, y + NODE_HEIGHT/2));
        }
        g2d.setColor(Color.BLACK);
        g2d.setFont(new Font("Arial", Font.BOLD, 16));
        g2d.drawString("Поколение " + generation, 30, y + NODE_HEIGHT/2 + 5);
        g2d.setColor(new Color(200, 200, 200, 120));
        g2d.setStroke(new BasicStroke(2, BasicStroke.CAP_BUTT, BasicStroke.JOIN_BEVEL,
                0, new float[]{8, 8}, 0));
        g2d.drawLine(startX - 30, y + NODE_HEIGHT/2,
                startX + totalWidth + 30, y + NODE_HEIGHT/2);
    }
}
private void drawHumanNode(Graphics2D g2d, Human human, int x, int y) {
    Color bgColor;
    if (!human.alive) {
        bgColor = new Color(200, 200, 200, 220);
    } else if (human == engine.getSelectedHuman()) {
        bgColor = new Color(255, 255, 150, 240);
    } else if (human.isMale) {
        bgColor = new Color(173, 216, 230, 240);
    } else {
        bgColor = new Color(255, 182, 193, 240);
    }
    g2d.setColor(new Color(100, 100, 100, 80));
    g2d.fillRoundRect(x + 3, y + 3, NODE_WIDTH, NODE_HEIGHT, 15, 15);
```

```
g2d.setColor(bgColor);
g2d.fillRoundRect(x, y, NODE_WIDTH, NODE_HEIGHT, 15, 15);
g2d.setColor(human == engine.getSelectedHuman() ? Color.RED :
        (human.isFounder() ? new Color(0, 100, 0) : Color.BLACK));
g2d.setStroke(new BasicStroke(human == engine.getSelectedHuman() ? 3.0f :
        (human.isFounder() ? 2.5f : 2.0f)));
g2d.drawRoundRect(x, y, NODE_WIDTH, NODE_HEIGHT, 15, 15);
g2d.setFont(new Font("Arial", Font.PLAIN, 11));
String displayName = human.name;
if (displayName.length() > 18) {
    displayName = displayName.substring(0, 16) + "...";
}
g2d.drawString(displayName, x + 8, y + 18);
String info = human.getGenderSymbol();
if (human.alive) {
    info += " " + human.age + " лет";
} else {
    if (human.deathAge == -1) {
        info += " ✝️ ? лет";
    } else {
        info += " ✝️ " + human.deathAge + " лет";
    }
}
g2d.drawString(info, x + 8, y + 35);
g2d.setColor(Color.DARK_GRAY);
g2d.drawString("ID:" + human.id + " Gen:" + human.generation, x + 8, y + 52);
int fitnessBar = (int)(human.fitness * (NODE_WIDTH - 20));
if (human.fitness > 0.7) {
    g2d.setColor(new Color(0, 180, 0));
} else if (human.fitness > 0.4) {
    g2d.setColor(new Color(255, 165, 0));
} else {
    g2d.setColor(Color.RED);
}
```

```
    g2d.fillRect(x + 8, y + NODE_HEIGHT - 10, fitnessBar, 6);
    g2d.setColor(Color.BLACK);
    g2d.setStroke(new BasicStroke(1.0f));
    g2d.drawRect(x + 8, y + NODE_HEIGHT - 10, NODE_WIDTH - 20, 6);
    g2d.setColor(Color.BLACK);
    g2d.setFont(new Font("Arial", Font.BOLD, 10));
    g2d.drawString(String.format("%.0f%%", human.fitness * 100),
            x + NODE_WIDTH - 30, y + NODE_HEIGHT - 4);
}
@Override
public void stateChanged(ChangeEvent e) {
    markBufferDirty();
    if (optimizeRendering) {
        SwingUtilities.invokeLater(() -> {
            lastRepaintCount++;
            if (lastRepaintCount > 5) {
                repaint();
                lastRepaintCount = 0;
            }
        });
    } else {
        repaint();
    }
}
public void optimizedRepaint() {
    if (optimizeRendering) {
        lastRepaintCount++;
        if (lastRepaintCount > 2) {
            repaint();
            lastRepaintCount = 0;
        }
    } else {
        repaint();
    }
```

```
    }
    @Override
    public void setBounds(int x, int y, int width, int height) {
        super.setBounds(x, y, width, height);
        markBufferDirty();
        if (fitToScreenButton != null) {
            fitToScreenButton.getParent().setBounds(10, 10, 180, 160);
        }
    }
    public void setOptimizeRendering(boolean optimize) {
        this.optimizeRendering = optimize;
        markBufferDirty();
        repaint();
    }
    private class Connection {
        Human parent;
        Human child;
        Color color;
        String type;
        Connection(Human parent, Human child, Color color, String type) {
            this.parent = parent;
            this.child = child;
            this.color = color;
            this.type = type;
        }
    }
}
```

Файл PopulationPanel.java

Код:

```
import javax.swing.*;
import javax.swing.event.*;
import javax.swing.table.*;
import java.awt.*;
```

```
import java.util.List;
import java.util.concurrent.atomic.AtomicBoolean;
class PopulationPanel extends JPanel implements ChangeListener {
    private EvolutionEngine engine;
    private JTable populationTable;
    private PopulationTableModel tableModel;
    private JLabel statsLabel;
    private AtomicBoolean updateInProgress = new AtomicBoolean(false);
    private long lastUpdateTime = 0;
    public PopulationPanel(EvolutionEngine engine) {
        this.engine = engine;
        engine.addChangeListener(this);
        setLayout(new BorderLayout());
        setBorder(BorderFactory.createTitledBorder("Текущая популяция"));
        setPreferredSize(new Dimension(300, 800));
        initializeComponents();
    }
    private void initializeComponents() {
        tableModel = new PopulationTableModel();
        populationTable = new JTable(tableModel);
        populationTable.setRowHeight(25);
        populationTable.setAutoCreateRowSorter(true);
        populationTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        populationTable.setAutoResizeMode(JTable.AUTO_RESIZE_OFF);
        UIManager.put("Table.ascendingSortIcon", null);
        UIManager.put("Table.descendingSortIcon", null);
        TableColumnModel columnModel = populationTable.getColumnModel();
        columnModel.getColumn(0).setPreferredWidth(40);  // ID
        columnModel.getColumn(1).setPreferredWidth(120); // Имя
        columnModel.getColumn(2).setPreferredWidth(30);  // Пол
        columnModel.getColumn(3).setPreferredWidth(40);  // Возр
        columnModel.getColumn(4).setPreferredWidth(50);  // Gen
        columnModel.getColumn(5).setPreferredWidth(60);  // Приспособленность
        columnModel.getColumn(5).setCellRenderer(new DefaultTableCellRenderer() {
```

```
    @Override
    public Component getTableCellRendererComponent(JTable table, Object value,
                                                   boolean isSelected, boolean hasFocus, int row, int column) {
        Component c = super.getTableCellRendererComponent(table, value,
                isSelected, hasFocus, row, column);
        if (!isSelected && value instanceof Double) {
            double fitness = (Double) value;
            if (fitness > 0.7) {
                c.setBackground(new Color(200, 255, 200));
            } else if (fitness > 0.4) {
                c.setBackground(new Color(255, 255, 200));
            } else {
                c.setBackground(new Color(255, 200, 200));
            }
        } else if (!isSelected) {
            c.setBackground(Color.WHITE);
        }
        return c;
    }
});
populationTable.setDefaultRenderer(Object.class, new DefaultTableCellRenderer() {
    @Override
    public Component getTableCellRendererComponent(JTable table, Object value,
                                                   boolean isSelected, boolean hasFocus, int row, int column) {
        Component c = super.getTableCellRendererComponent(table, value,
                isSelected, hasFocus, row, column);
        if (!isSelected) {
            int modelRow = table.convertRowIndexToModel(row);
            Human human = tableModel.getHumanAt(modelRow);
            if (human != null) {
                if (!human.alive) {
                    c.setBackground(Color.LIGHT_GRAY);
                } else if (human.isMale) {
                    c.setBackground(new Color(220, 240, 255));
```

```
                } else {
                    c.setBackground(new Color(255, 220, 230));
                }
            }
        }
        return c;
    }
});
populationTable.getSelectionModel().addListSelectionListener(e -> {
    if (!e.getValueIsAdjusting()) {
        int row = populationTable.getSelectedRow();
        if (row >= 0) {
            int modelRow = populationTable.convertRowIndexToModel(row);
            Human human = tableModel.getHumanAt(modelRow);
            engine.selectHuman(human);
        }
    }
});
JScrollPane scrollPane = new JScrollPane(populationTable);
statsLabel = new JLabel(" ");
statsLabel.setFont(new Font("Arial", Font.PLAIN, 11));
JPanel statsPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
statsPanel.add(statsLabel);
add(scrollPane, BorderLayout.CENTER);
add(statsPanel, BorderLayout.SOUTH);
updateStatistics();
}
private void updateStatistics() {
long currentTime = System.currentTimeMillis();
if (currentTime - lastUpdateTime < 100) {
    return;
}
List<Human> alive = engine.getAlivePeople();
List<Human> all = engine.getAllPeople();
```

```
        long males = alive.stream().filter(h -> h.isMale).count();
        long females = alive.size() - males;
        double avgAge = alive.stream().mapToInt(h -> h.age).average().orElse(0);
        double avgFitness = engine.getAverageFitness();
        double bestFitness = engine.getBestFitness();
        statsLabel.setText(String.format(
                "<html>Живых: <b>%d</b> (♂%d ♀%d) | Всего: <b>%d</b><br>" +
                                        "Ср.возр: <b>%.1f</b> | Ср.присп: <b>%.2f</b> | Лучшая:
<b>%.2f</b></html>",
                alive.size(), males, females, all.size(), avgAge, avgFitness, bestFitness
        ));
        lastUpdateTime = currentTime;
    }
    @Override
    public void stateChanged(ChangeEvent e) {
        if (updateInProgress.compareAndSet(false, true)) {
            try {
                tableModel.updateData(engine.getAllPeople());
                updateStatistics();
            } finally {
                updateInProgress.set(false);
            }
        }
    }
    static class PopulationTableModel extends AbstractTableModel {
        private List<Human> population;
        private final String[] columns = {"ID", "Имя", "Пол", "Возр", "Gen", "Присп.", "Статус"};

        public PopulationTableModel() {
            this.population = List.of();
        }
        public void updateData(List<Human> population) {
            SwingUtilities.invokeLater(() -> {
                this.population = population;
```

```
        fireTableDataChanged();
    });
}
public Human getHumanAt(int row) {
    return population.get(row);
}
@Override
public int getRowCount() {
    return population.size();
}
@Override
public int getColumnCount() {
    return columns.length;
}
@Override
public String getColumnName(int column) {
    return columns[column];
}
@Override
public Object getValueAt(int row, int column) {
    Human human = population.get(row);
    return switch (column) {
        case 0 -> human.id;
        case 1 -> human.name;
        case 2 -> human.getGenderSymbol();
        case 3 -> human.age;
        case 4 -> human.generation;
        case 5 -> human.fitness;
        case 6 -> human.alive ? "Жив" : "Умер";
        default -> "";
    };
}
@Override
public Class<?> getColumnClass(int columnIndex) {
```

```
            return switch (columnIndex) {
                case 0 -> Integer.class;
                case 1 -> String.class;
                case 2 -> String.class;
                case 3 -> Integer.class;
                case 4 -> Integer.class;
                case 5 -> Double.class;
                case 6 -> String.class;
                default -> Object.class;
            };
        }
    }
}
```